

ST. XAVIER'S COLLEGE

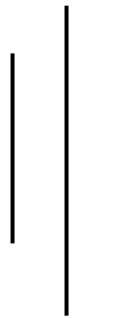
Maitighar, Kathmandu

(Affiliated to Tribhuvan University)



LAB REPORT ON

DATA WAREHOUSING AND DATA MINING (CSC420)



SUBMITTED BY:

Name: Aayush Acharya

Faculty/Semester: CSIT/7th Sem

Roll No.: 022BSCIT004

SUBMITTED TO:

Er. Anuj Shrestha

Lecturer

Dept. of Computer Science



ST. XAVIER'S COLLEGE

MAITIGHAR, KATHMANDU

Data Warehousing and Data Mining (CSC420) Lab Index

Name: Aayush Acharya

College roll number: 022bscit004

S.N.	Lab Title
1	Preprocessing of primary and secondary datasets containing dirty data.
2	Designing various Data Warehouse Schema (Star Schema, Snowflake Schema, Fact Constellation) using MySQL query.
3	Programmatically illustrate K nearest neighbour algorithm.
4	Use ID3 and J48 filters in WEKA to design decision trees for a primary dataset. Assume a suitable cross validation fold, also visualize the tree using J48.
5	Create a decision tree model using python programming language.
6	Illustrate Multilayer Perceptron in WEKA.
7	Use minimum support of 0.3 when generating frequent patterns using Apriori and FPGrowth filters of WEKA on a suitable primary dataset. Then, find strong association rules using minimum confidence of 0.85.
8	Programmatically demonstrate a binary classifier using Naive Bayes algorithm.
9	Using python programming implement K-means clustering, agglomerative hierarchical clustering and DBScan.
10	Apply agglomerative hierarchical clustering filter of WEKA on a suitable dataset.

Lab Number: 1

Title: Preprocessing of Primary and Secondary Datasets Containing Dirty Data

Objective: To understand and apply data preprocessing techniques on primary and secondary datasets that contain dirty data, in order to prepare clean and consistent data for further analysis.

IDE/Tools Used: MS Excel, Python (Pandas library)

Theory:

Data Preprocessing

Data preprocessing is a crucial step in data analysis and machine learning pipelines. It involves transforming raw data into a clean and structured format suitable for analysis. The goal is to ensure data consistency, remove errors, and handle missing or irrelevant information.

Types of Dirty Data

Dirty data refers to data that contains errors, inconsistencies, or missing values. Common types include:

1. **Missing Data:** Incomplete records where values are absent.
2. **Duplicate Data:** Repeated entries that can skew analysis.
3. **Inconsistent Data:** Variations in data formats or units (e.g., "kg" vs. "pounds").
4. **Outliers:** Extreme values that deviate significantly from other observations.
5. **Irrelevant Data:** Data that does not contribute to the analysis objective.

Datasets

1. **Primary Dataset:** The main dataset collected directly from the source, such as surveys, experiments, or observations.
2. **Secondary Dataset:** Data collected by others, often used to complement the primary dataset. Examples include public datasets, reports, or data from external organizations.

Preprocessing Techniques

Data preprocessing involves various techniques to handle missing or inconsistent data. Missing data can be addressed by dropping rows or columns with null values or by imputing them using statistical methods like mean, median, or mode. Duplicate entries are identified and removed to ensure data integrity. Standardization and normalization are applied to maintain consistent data formats and scale numerical values for uniformity. Outliers, which are extreme deviations, are detected and managed using statistical methods or domain-specific knowledge to prevent skewed analysis. These techniques collectively ensure clean, reliable, and analyzable datasets.

Implementation:

```
import pandas as pd
```

```
# Load the primary dataset
```

```
primary_file_path = "/Lab 1/barcelona_players.csv"
```

```
primary_df = pd.read_csv(primary_file_path)
```

```
# Load the secondary dataset
```

```
secondary_file_path = "/Lab 1/secondary_dataset.csv"
```

```
secondary_df = pd.read_csv(secondary_file_path)
```

```
# Clean the primary dataset
```

```
primary_df_cleaned = primary_df.dropna()
```

```
# Clean the secondary dataset
```

```
secondary_df_cleaned = secondary_df.dropna()
```

```
# Combine the cleaned datasets
```

```
combined_df = pd.merge(primary_df_cleaned, secondary_df_cleaned, left_on="player_id",  
right_on="match_id", how="outer")
```

```
print("Cleaned Dataset:")
```

```
print(combined_df)
```

Output

Primary dirty dataset:

	A	B	C	D	E	F
1	player_id	player_name	position	goals	assists	club
2	1	Messi	Forward	30	12	Barcelona
3	2	Xavi	Midfielder	10	15	Barcelona
4	3	Iniesta	Midfielder	5		Barcelona
5	4	Puyol	Defender		0	Barcelona
6	5	Busquets		3	8	Barcelona
7	6	Neymar	Forward	20	10	
8	7			15		Barcelona

Secondary Dataset:

	A	B	C	D	E
1	match_id	opponent	goals_scored	goals_conceded	
2	101	Real Madrid	2	1	
3	102	Atletico Madrid	1		
4	103	Sevilla		0	
5	104	Valencia	3	2	
6	105		0	1	

Cleaned Dataset:

```
PS D:\Aayush_Acharya\7th Sem\DWDM> & "C:/Users/N I T R O/AppData/Local/Programs/Python/Python313/python.exe" "d:/Aayush_Acharya/7th Sem/DWDM/Lab 1/preprocessing_code.py"
Cleaned Dataset:
   player_id player_name position ... opponent goals_scored goals_conceded
0         1.0      Messi   Forward ...        NaN          NaN          NaN
1         2.0       Xavi  Midfielder ...        NaN          NaN          NaN
2         NaN        NaN      NaN ...   Real Madrid          2.0          1.0
3         NaN        NaN      NaN ...    Valencia          3.0          2.0

[4 rows x 10 columns]
```

Discussion:

The preprocessing of the primary and secondary datasets involved handling missing values by dropping incomplete rows to ensure data consistency. The cleaned datasets were then merged into a single dataset, aligning records based on relevant keys (player_id and match_id). This approach ensures that the final dataset is free from inconsistencies and ready for further analysis. While dropping rows with missing values is a straightforward method, it may lead to data loss, which should be considered based on the dataset's size and importance of missing entries.

Conclusion:

The preprocessing techniques applied successfully cleaned and combined the datasets, resulting in a unified and consistent dataset. This process highlights the importance of addressing dirty data to improve data quality and reliability for analysis. Future steps could involve exploring imputation methods to handle missing values without data loss..

Lab Number: 2

Title: Designing Data Warehouse Schema (Star Schema, Snowflake Schema, Fact Constellation) Using MySQL Query

Objective: To design and implement different types of data warehouse schemas i.e. Star Schema, Snowflake Schema, and Fact Constellation using MySQL queries.

IDE/Tools Used: VS Code,dbdiagram.io

Theory:

Data Warehouse

A Data Warehouse is a centralized repository that stores integrated data from multiple sources. It is designed for query and analysis rather than transaction processing. Data warehouses support business intelligence activities, such as reporting, analytics, and decision-making.

Star Schema

Star Schema is the simplest form of a data warehouse schema. It consists of a central fact table connected to multiple dimension tables. The fact table contains quantitative data (e.g., sales, revenue), while dimension tables provide descriptive attributes (e.g., product, time, location).

Snowflake Schema

The Snowflake Schema builds upon the Star Schema by normalizing the dimension tables into smaller sub-dimensions. This normalization reduces redundancy and improves data integrity, making it more suitable for large datasets. While the Snowflake Schema is more complex and requires additional joins during queries, it is better equipped to handle complex analytical tasks. The normalized structure ensures that updates and changes to the data are easier to manage, as the data is stored in a more organized and consistent manner. This schema is often used in scenarios where data integrity and storage optimization are prioritized over query performance.

Fact Constellation Schema

The Fact Constellation Schema, also known as the Galaxy Schema, is designed to support multiple fact tables that share common dimension tables. This schema is used in complex analytical environments where multiple business processes need to be analyzed simultaneously. By sharing dimensions, the Fact Constellation Schema reduces redundancy and provides a flexible structure for analyzing data from different perspectives. However, the increased complexity of this schema requires careful design and management to ensure efficient query performance. It is best suited for large-scale data warehouses where diverse analytical requirements need to be addressed.

Implementation:

1) Star Schema:

```
CREATE DATABASE SALES_STAR;
```

```
USE SALES_STAR;
```

```
CREATE TABLE sales_source (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(255),  
    category VARCHAR(255),  
    sale_date DATE,  
    store_location VARCHAR(255),  
    units_sold INT,  
    revenue DECIMAL(10, 2)  
);
```

```
INSERT INTO sales_source (product_name, category, sale_date, store_location, units_sold, revenue)  
VALUES  
( 'Gaming Laptop', 'Electronics', '2026-04-01', 'San Francisco', 8, 12000.00),  
( 'Smartwatch', 'Wearables', '2026-04-10', 'Seattle', 12, 6000.00),  
( 'Wireless Earbuds', 'Audio', '2026-04-15', 'Austin', 20, 4000.00);
```

```
CREATE TABLE time_dim (  
    t_id INT AUTO_INCREMENT PRIMARY KEY,  
    sale_date DATE  
);
```

```
INSERT INTO time_dim (sale_date)  
SELECT DISTINCT sale_date FROM sales_source;
```

```
CREATE TABLE product_dim (  
    p_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(255),  
    category VARCHAR(255)  
);
```

```
INSERT INTO product_dim (product_name, category)  
SELECT DISTINCT product_name, category FROM sales_source;
```

```
CREATE TABLE location_dim (  
    l_id INT AUTO_INCREMENT PRIMARY KEY,  
    store_location VARCHAR(255)  
);
```

```
INSERT INTO location_dim (store_location)  
SELECT DISTINCT store_location FROM sales_source;
```

```
CREATE TABLE sales_fact (  
    t_id INT,  
    p_id INT,  
    l_id INT,  
    units_sold INT,
```



```
revenue DECIMAL(10, 2),
FOREIGN KEY (t_id) REFERENCES time_dim(t_id),
FOREIGN KEY (p_id) REFERENCES product_dim(p_id),
FOREIGN KEY (l_id) REFERENCES location_dim(l_id)
);

INSERT INTO sales_fact (t_id, p_id, l_id, units_sold, revenue)
SELECT t.t_id, p.p_id, l.l_id, s.units_sold, s.revenue
FROM sales_source s
LEFT JOIN time_dim t ON t.sale_date = s.sale_date
LEFT JOIN product_dim p ON p.product_name = s.product_name AND p.category = s.category
LEFT JOIN location_dim l ON l.store_location = s.store_location;

SELECT sf.*, t.sale_date
FROM sales_fact sf
LEFT JOIN time_dim t ON t.t_id = sf.t_id
WHERE YEAR(t.sale_date) = 2026;
```

2) Snowflake Schema:

```
CREATE DATABASE FOOTBALL_SNOWFLAKE;
USE FOOTBALL_SNOWFLAKE;
-- Central Fact Table
CREATE TABLE match_fact (
    match_id INT AUTO_INCREMENT PRIMARY KEY,
    date_id INT,
    team_id INT,
    opponent_id INT,
    stadium_id INT,
    goals_scored INT,
    goals_conceded INT,
    points INT,
    FOREIGN KEY (date_id) REFERENCES date_dim(date_id),
    FOREIGN KEY (team_id) REFERENCES team_dim(team_id),
    FOREIGN KEY (opponent_id) REFERENCES team_dim(team_id),
    FOREIGN KEY (stadium_id) REFERENCES stadium_dim(stadium_id)
);
-- Date Dimension (Normalized)
CREATE TABLE date_dim (
    date_id INT AUTO_INCREMENT PRIMARY KEY,
    match_date DATE,
    month_id INT,
    day_of_week VARCHAR(20),
    FOREIGN KEY (month_id) REFERENCES month_dim(month_id)
);
CREATE TABLE month_dim (
    month_id INT AUTO_INCREMENT PRIMARY KEY,
    month_name VARCHAR(20),
    year INT
);
-- Team Dimension (Normalized)
CREATE TABLE team_dim (
    team_id INT AUTO_INCREMENT PRIMARY KEY,
    team_name VARCHAR(255),
    manager_name VARCHAR(255),
    league_id INT,
    FOREIGN KEY (league_id) REFERENCES league_dim(league_id)
);
CREATE TABLE league_dim (
    league_id INT AUTO_INCREMENT PRIMARY KEY,
    league_name VARCHAR(255),
    country VARCHAR(255)
);
-- Stadium Dimension (Already Normalized)
CREATE TABLE stadium_dim (
    stadium_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    stadium_name VARCHAR(255),  
    city VARCHAR(255),  
    capacity INT  
);
```

```
-- Insert Sample Data
```

```
INSERT INTO month_dim (month_name, year) VALUES  
('April', 2026);
```

```
INSERT INTO date_dim (match_date, month_id, day_of_week) VALUES  
('2026-04-01', 1, 'Wednesday'),  
('2026-04-10', 1, 'Friday');
```

```
INSERT INTO league_dim (league_name, country) VALUES  
('La Liga', 'Spain');
```

```
INSERT INTO team_dim (team_name, manager_name, league_id) VALUES  
('Barcelona', 'Xavi Hernandez', 1),  
('Real Madrid', 'Carlo Ancelotti', 1);
```

```
INSERT INTO stadium_dim (stadium_name, city, capacity) VALUES  
('Camp Nou', 'Barcelona', 99354),  
('Santiago Bernabeu', 'Madrid', 81044);
```

```
INSERT INTO match_fact (date_id, team_id, opponent_id, stadium_id, goals_scored, goals_conceded,  
points) VALUES  
(1, 1, 2, 1, 2, 1, 3),  
(2, 2, 1, 2, 1, 2, 0);
```

3) Fact Constellation:

```
# Create database
CREATE DATABASE FOOTBALL_CONSTELLATION;

USE FOOTBALL_CONSTELLATION;

-- Match Fact Table
CREATE TABLE match_fact (
    match_id INT AUTO_INCREMENT PRIMARY KEY,
    date_id INT,
    team_id INT,
    opponent_id INT,
    stadium_id INT,
    goals_scored INT,
    goals_conceded INT,
    points INT,
    FOREIGN KEY (date_id) REFERENCES date_dim(date_id),
    FOREIGN KEY (team_id) REFERENCES team_dim(team_id),
    FOREIGN KEY (opponent_id) REFERENCES team_dim(team_id),
    FOREIGN KEY (stadium_id) REFERENCES stadium_dim(stadium_id)
);

-- Player Performance Fact Table
CREATE TABLE player_fact (
    player_id INT,
    match_id INT,
    goals INT,
    assists INT,
    minutes_played INT,
    FOREIGN KEY (player_id) REFERENCES player_dim(player_id),
    FOREIGN KEY (match_id) REFERENCES match_fact(match_id)
);

-- Date Dimension
CREATE TABLE date_dim (
    date_id INT AUTO_INCREMENT PRIMARY KEY,
    match_date DATE,
    year INT,
    month VARCHAR(20),
    day_of_week VARCHAR(20)
);

-- Team Dimension
CREATE TABLE team_dim (
    team_id INT AUTO_INCREMENT PRIMARY KEY,
    team_name VARCHAR(255),
    manager_name VARCHAR(255),
    league VARCHAR(255)
);
```

-- Stadium Dimension

```
CREATE TABLE stadium_dim (  
    stadium_id INT AUTO_INCREMENT PRIMARY KEY,  
    stadium_name VARCHAR(255),  
    city VARCHAR(255),  
    capacity INT  
);
```

-- Player Dimension

```
CREATE TABLE player_dim (  
    player_id INT AUTO_INCREMENT PRIMARY KEY,  
    player_name VARCHAR(255),  
    position VARCHAR(50),  
    nationality VARCHAR(50)  
);
```

-- Insert Sample Data

```
INSERT INTO player_dim (player_name, position, nationality) VALUES  
( 'Lionel Messi', 'Forward', 'Argentina'),  
( 'Pedri', 'Midfielder', 'Spain');
```

```
INSERT INTO player_fact (player_id, match_id, goals, assists, minutes_played) VALUES  
(1, 1, 2, 1, 90),  
(2, 1, 0, 1, 85);
```

Screenshots of generated Data warehouse schema:

Fig1: Star Schema using SQL

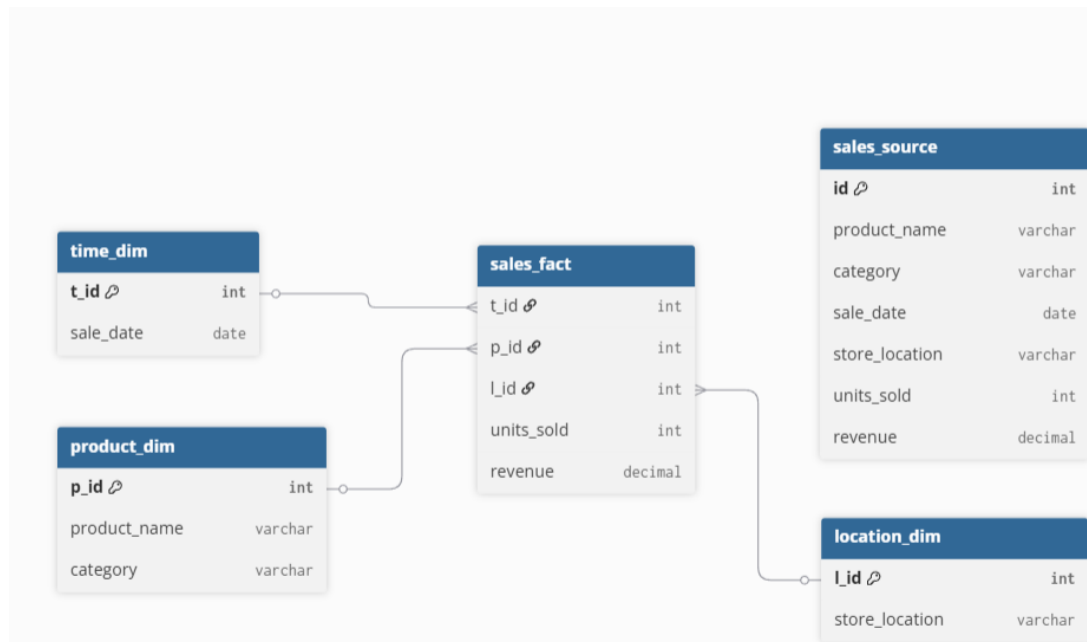


Fig2: Snowflake Schema using SQL

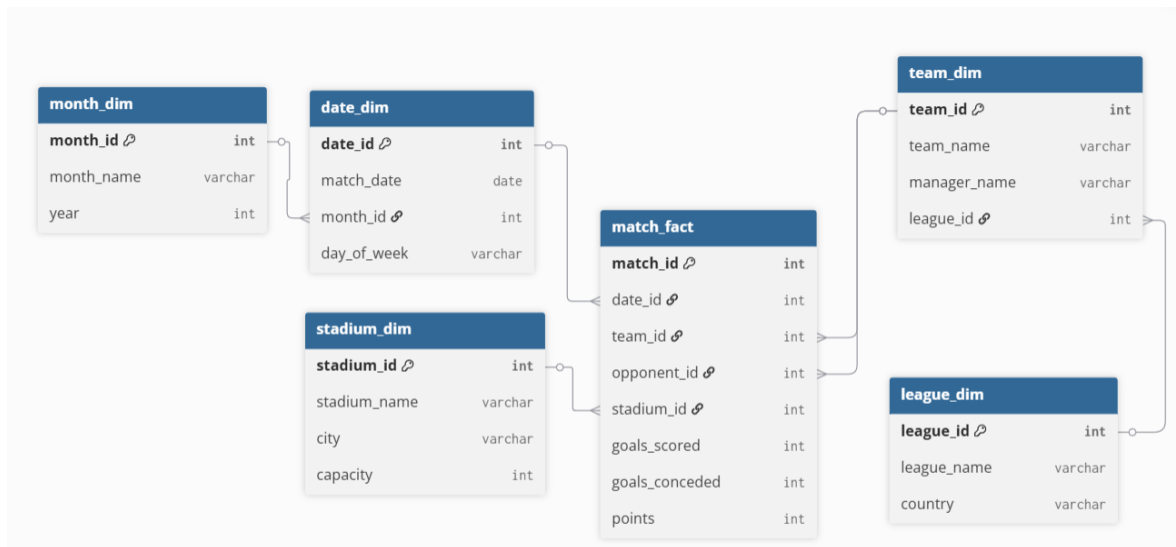
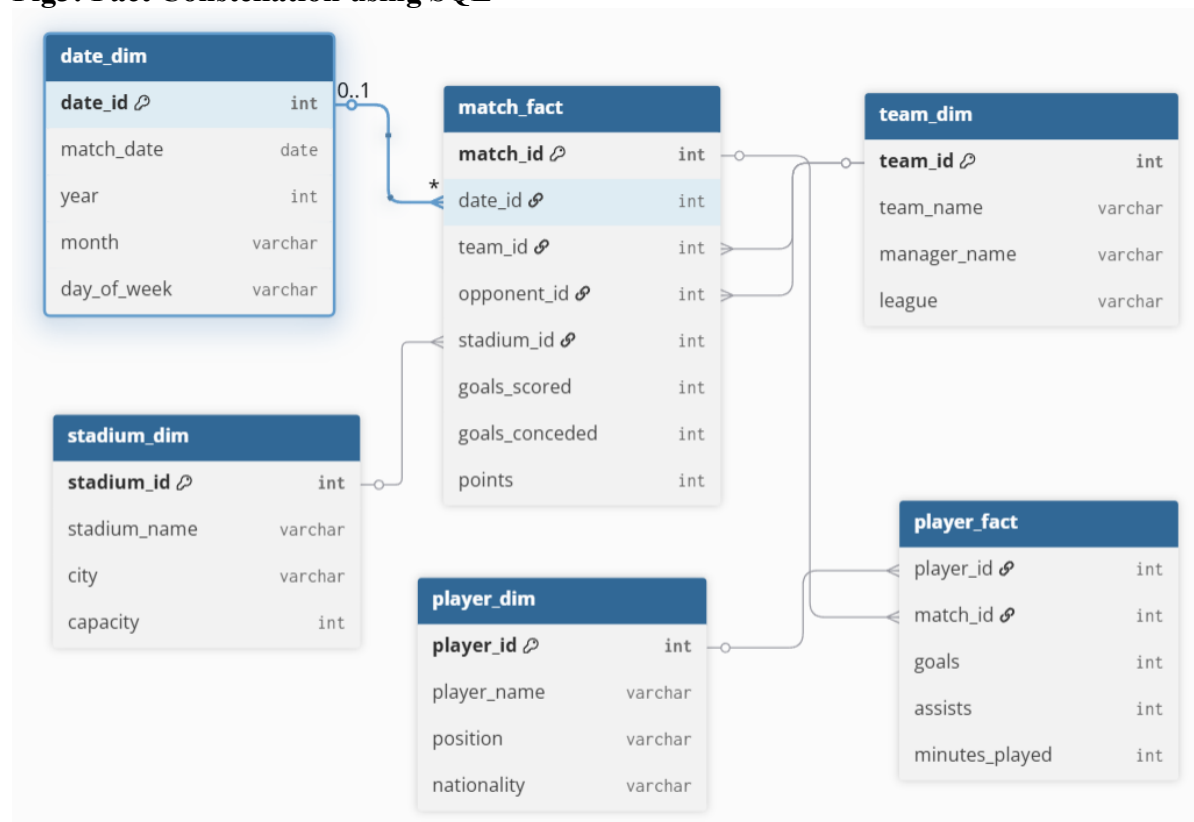


Fig3: Fact Constellation using SQL



Discussion:

The purpose of this lab was to design and implement three data warehouse schemas using SQL queries in VS Code with MySQL as the database platform. The schemas were visualized using dbdiagram.io for clear representation of relationships and foreign keys. The Star Schema was implemented first, with a central fact table directly referencing flat dimension tables. Next, the Snowflake Schema normalized the dimension tables, reducing redundancy but requiring more joins. Finally, the Fact Constellation Schema was created with two fact tables sharing common dimensions, making it suitable for analyzing multiple processes. Dummy data was inserted into source tables and transformed into fact and dimension tables using INSERT INTO ... SELECT queries.

Conclusion:

This lab demonstrated the design of three key data warehouse schemas. The Star Schema was simple and fast for queries, while the Snowflake Schema improved data integrity through normalization. The Fact Constellation Schema provided flexibility for analyzing multiple processes but was more complex. The choice of schema depends on the organization's needs, balancing performance, storage, and complexity.

Lab Number: 3

Title: Programmatically Illustrate K-Nearest Neighbour Algorithm

Objective: To implement the K-Nearest Neighbors (KNN) algorithm to classify a new data point based on similarity with labeled training data.

IDE/Tools Used: Python (numpy,Pandas, Matplotlib), Jupyter Notebook

Theory:

K-Nearest Neighbour (KNN) Algorithm

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

The K-Nearest Neighbors (KNN) algorithm is a simple, non-parametric, and supervised machine learning technique used for classification and regression tasks. It works on the principle that similar data points exist close to each other in a feature space. In this implementation, KNN is used for classification, where a new (test) data point is assigned a class label based on the majority class among its k nearest neighbors from the training dataset.

Euclidean distance is commonly used to measure similarity between points. The formula for Euclidean distance is:

KNN is easy to implement and effective for small datasets, but its performance depends on the choice of k and the scale of features.

Applications and Limitations

KNN finds many applications in various fields, including recommendation systems, pattern recognition tasks, and even medical diagnostics. The disadvantages of KNN include a relatively long time needed for predictions due to the necessity to calculate the distances to each element from the training dataset, sensitivity to the curse of dimensionality, and vulnerability to irrelevant features. Therefore, it is crucial to scale the features before employing KNN.

Implementation:

```
import numpy as np
from collections import Counter
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((np.array(point1) - np.array(point2))**2))

# Define KNN Prediction Function
def knn_predict(training_data, training_labels, test_point, k):
    distances = []
    for i in range(len(training_data)):
        dist = euclidean_distance(test_point, training_data[i])
        distances.append((dist, training_labels[i]))
    distances.sort(key=lambda x: x[0])
    k_nearest_labels = [label for _, label in distances[:k]]
    return Counter(k_nearest_labels).most_common(1)[0][0]
training_data = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8]]
training_labels = ['A', 'A', 'A', 'B', 'B']

# Test Data Point
test_point = [4, 7]
k = 3

# Predict Class
prediction = knn_predict(training_data, training_labels, test_point, k)
print("Test point:", test_point)
print("Predicted class:", prediction)

import matplotlib.pyplot as plt
import numpy as np

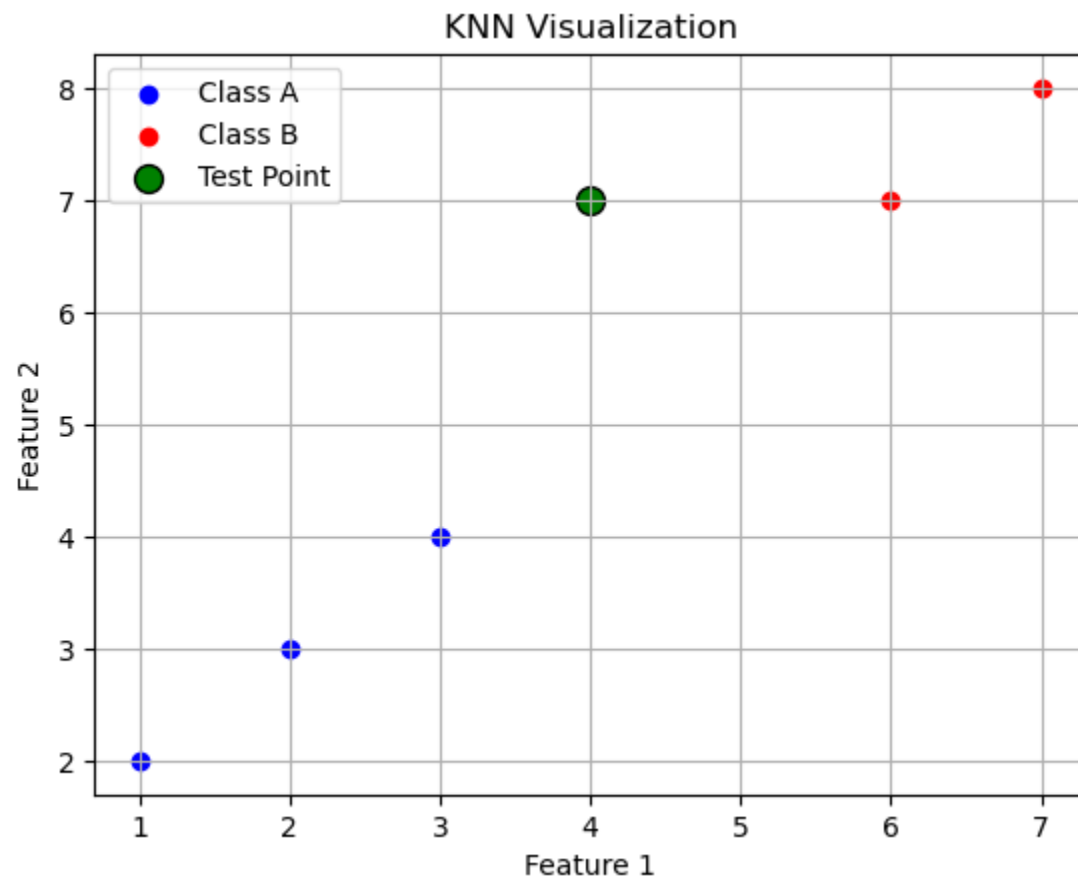
# Convert training data and labels to numpy arrays for easier handling
training_data_np = np.array(training_data)
training_labels_np = np.array(training_labels)

# Separate training data by class
class_A = training_data_np[training_labels_np == 'A']
class_B = training_data_np[training_labels_np == 'B']

# Plot training data
plt.scatter(class_A[:, 0], class_A[:, 1], color='blue', label='Class A')
plt.scatter(class_B[:, 0], class_B[:, 1], color='red', label='Class B')
# Plot test point
plt.scatter(test_point[0], test_point[1], color='green', label='Test Point', edgecolor='black', s=100)
# Add labels and legend
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('KNN Visualization')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Test point: [4, 7]  
Predicted class: B
```



Discussion:

In this lab, we implemented the K-Nearest Neighbors (KNN) algorithm using Python in VS Code. The algorithm was applied to classify a test data point based on its similarity to labeled training data. The Euclidean distance metric was used to calculate the distance between the test point and all training points. The top-k nearest neighbors were selected, and the majority class among these neighbors determined the predicted class. The implementation demonstrated the simplicity and effectiveness of KNN for small datasets.

Conclusion:

This lab provided hands-on experience with the KNN algorithm, showcasing its ability to classify data points based on proximity to labeled examples. While KNN is easy to implement and interpret, its performance depends on the choice of k and the scaling of features. The algorithm is computationally expensive for large datasets, but it remains a powerful tool for small-scale classification tasks.

Lab Number: 4

Title: ID3 and J48 Algorithm to Design Decision Trees Using a Primary Dataset and Visualizing the Tree Using J48 Algorithm

Objective: To classify a primary dataset using decision trees built with the ID3 and J48 algorithms in Weka, and to visualize the resulting tree using J48.

IDE/Tools Used: MS Excel, Weka

Theory:

Decision Tree

A decision tree is a tree structure that splits the dataset into smaller subsets according to some tests performed on the value of the input attributes. Each node of the decision tree is a test of the attributes, branches correspond to outcomes, while leaf nodes are assigned one of the possible class labels (regression). Decision trees are easy to interpret and use, and do not require much pre-processing.

Classification

Classification is the prediction of categories of data points according to their attributes. In decision tree classification, models learn rules to classify new data points on the basis of the knowledge gained from the analysis of training data. Classification is effective when using the right datasets and algorithms and properly tuning model parameters.

Primary Dataset

A primary dataset is data obtained by conducting an experiment, observation, or survey. In the case of decision tree classification, the data set should consist of attributes to be classified and the class labels. A dataset for use in classification tasks should be loaded into Weka in ARFF (attribute relation file format) and consist of instances and attributes.

Using Weka for Decision Trees

The Weka software implements two different algorithms to construct a decision tree: ID3 and J48. While the ID3 algorithm operates only with nominal attributes, the J48 algorithm is an implementation of the C4.5 algorithm and takes care of both categorical and numerical attributes. Moreover, the J48 algorithm copes with the presence of missing values and introduces the concept of pruning to mitigate the problem of overfitting. Visualizing the generated tree is possible for J48 algorithm only in Weka.

ID3 Algorithm

ID3 (Iterative Dichotomiser 3) is a decision tree algorithm invented by Ross Quinlan. For the construction of the decision tree, this algorithm utilizes the information gain measure. This

measure is based on entropy and calculates how much reduction in entropy is provided by the partition based on the specific attribute. In other words, this algorithm splits nodes based on an attribute that provides the maximum value of information gain. The splitting procedure stops when all instances of the data belong to the same category.

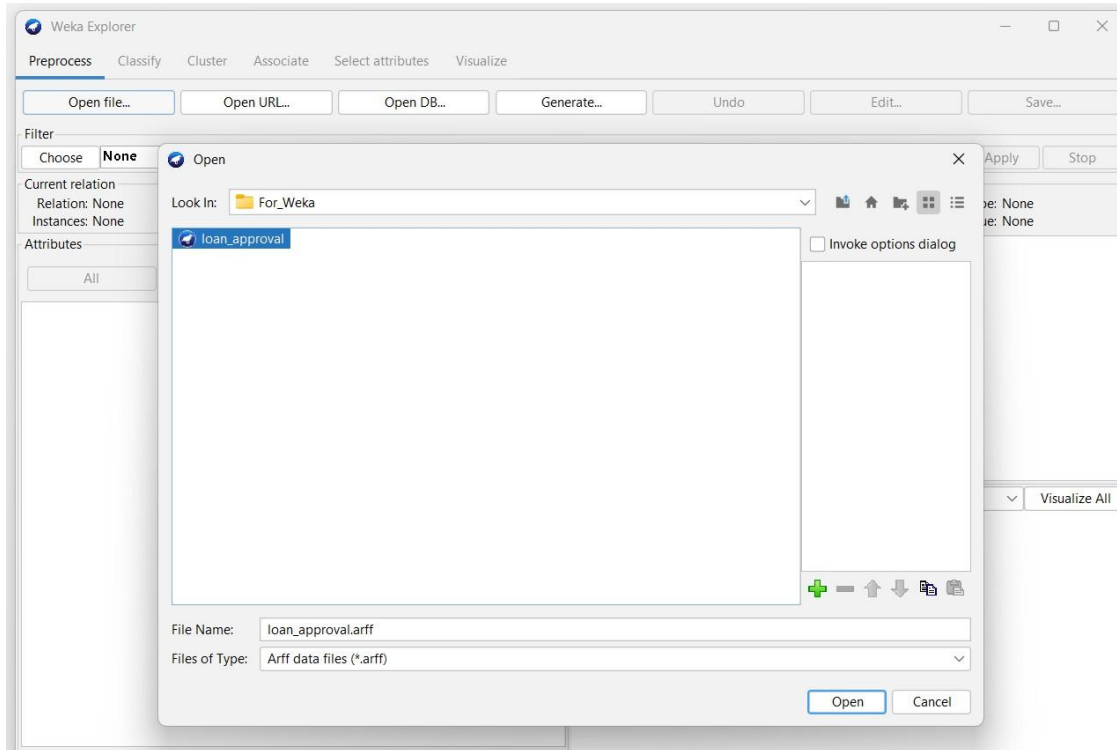
J48 Algorithm

The J48 classifier in Weka is the open-source version of C4.5 classifier coded in Java. The difference between J48 and ID3 classifiers lies in the method for selecting splitting attributes in a tree structure. While ID3 selects splitting attributes using the information gain technique, J48 chooses them through the gain ratio approach, which compensates for the problem faced by ID3 classifiers concerning its tendency to select high-value attributes. In addition, J48 can handle continuous attributes through their optimal splitting, missing attributes, and post-pruning. In Weka, when J48 is applied, one right-clicks on its output to view its decision tree graphically.

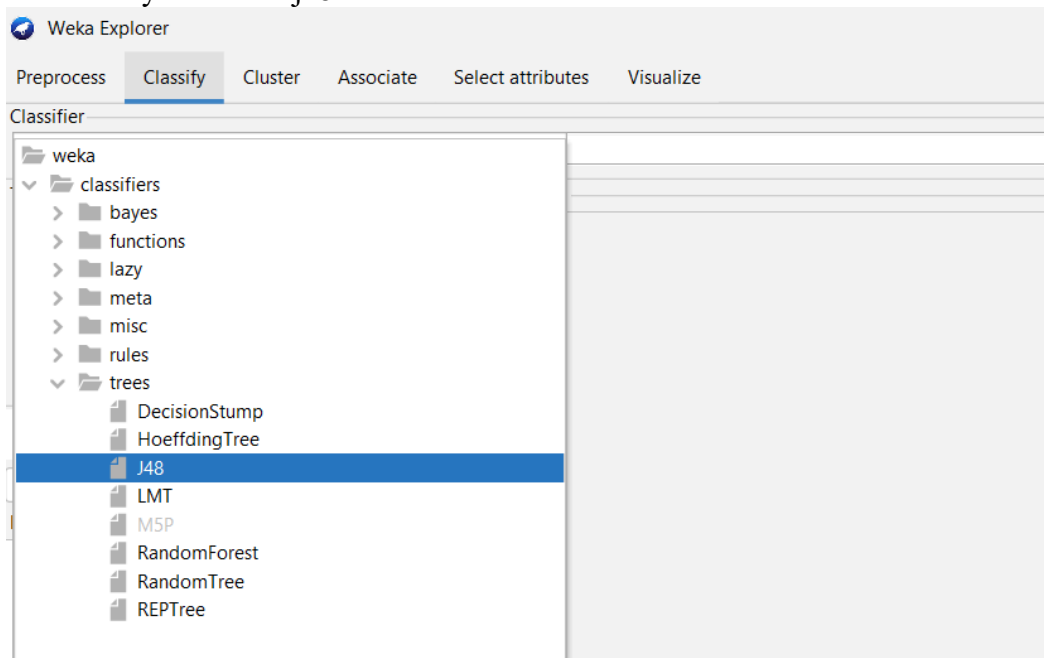
Implementation

1. J48 Classifier

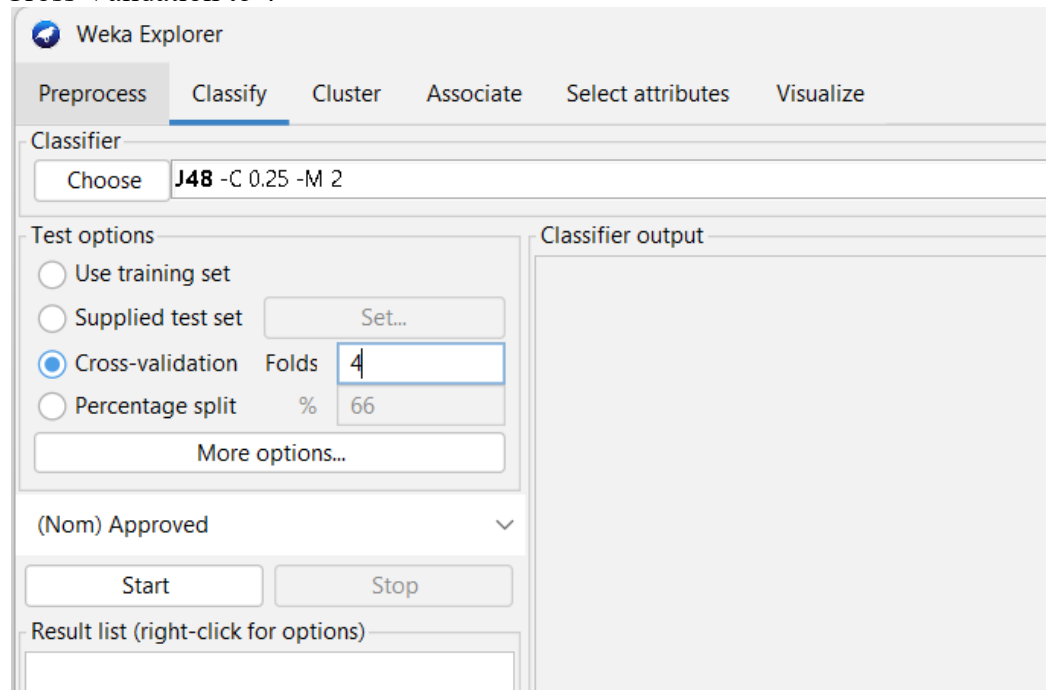
- Load the dataset:



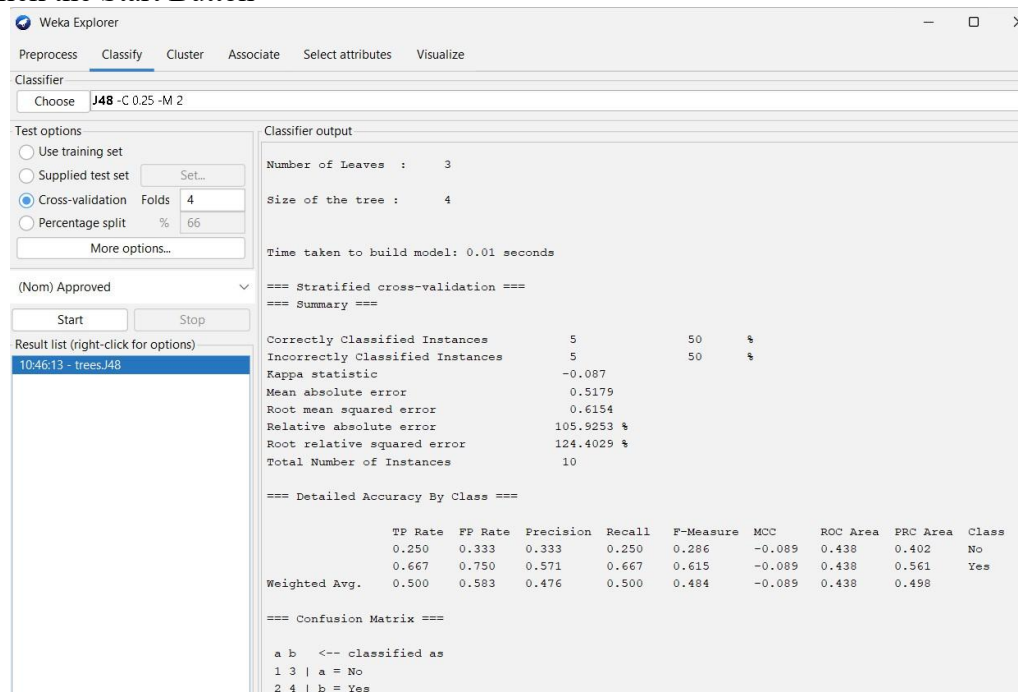
- In the classify tab select j48



- Set Cross Validation to 4



- Click the Start Button



- Now right click on treeJ48 and choose the Visualize tree

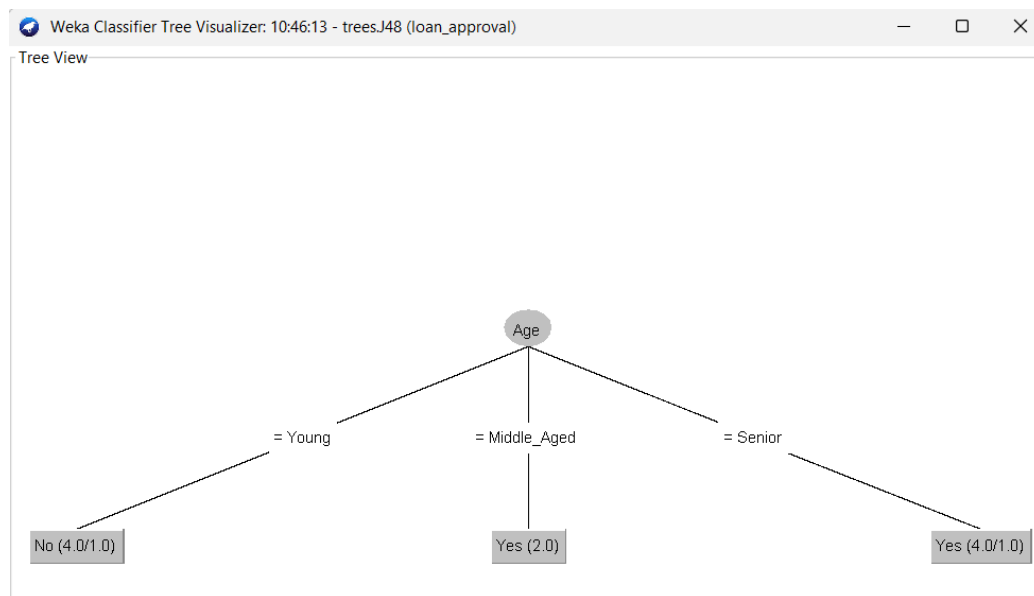
The screenshot shows the Weka Explorer interface. The 'Classifier' tab is active, displaying the 'J48 -C 0.25 -M 2' model. The 'Test options' section shows 'Cross-validation' with 'Folds' set to 4. The 'Classifier output' section displays the following information:

- Number of Leaves : 3
- Size of the tree : 4
- Time taken to build model: 0.01 seconds
- Stratified cross-validation summary:

Summary			
Correctly Classified Instances	5	50	%
Incorrectly Classified Instances	5	50	%
	-0.087		
	0.5179		
	0.6154		
	105.9253	%	
	124.4029	%	
	10		

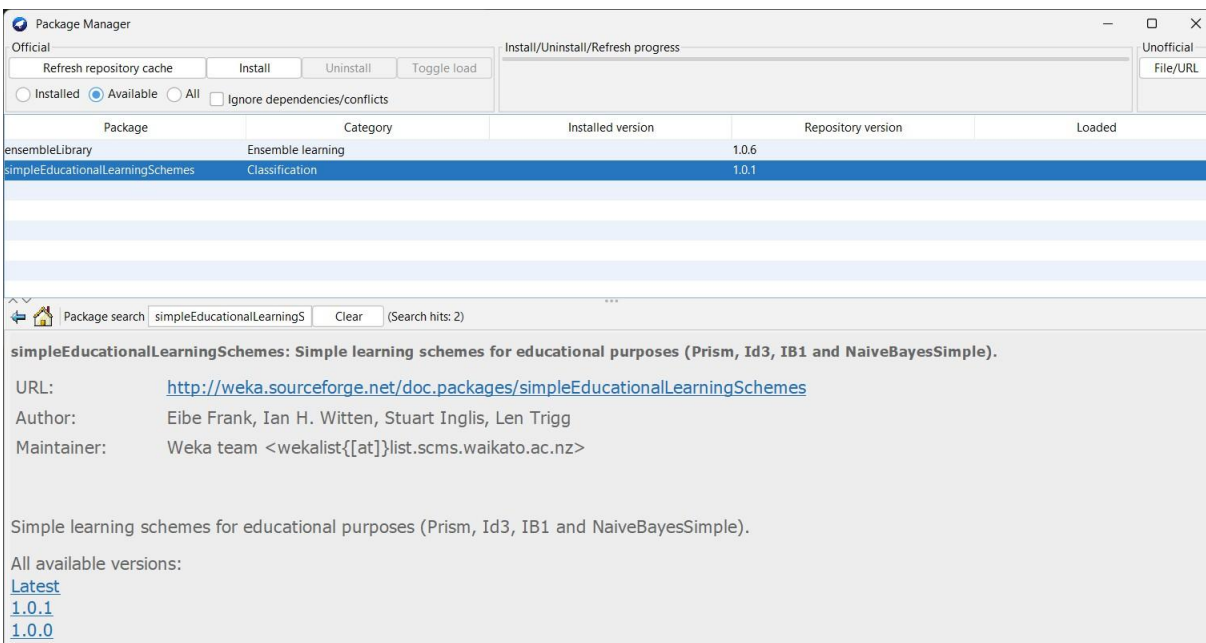
A right-click context menu is open over the 'Result list' (10:46:13 - trees.J48), with the 'Visualize tree' option selected. Other options include 'View in main window', 'View in separate window', 'Save result buffer', 'Delete result buffer(s)', 'Load model', 'Save model', 'Re-evaluate model on current test set', 'Re-apply this model's configuration', 'Visualize classifier errors', 'Visualize margin curve', 'Visualize threshold curve', 'Cost/Benefit analysis', and 'Visualize cost curve'.

Output:

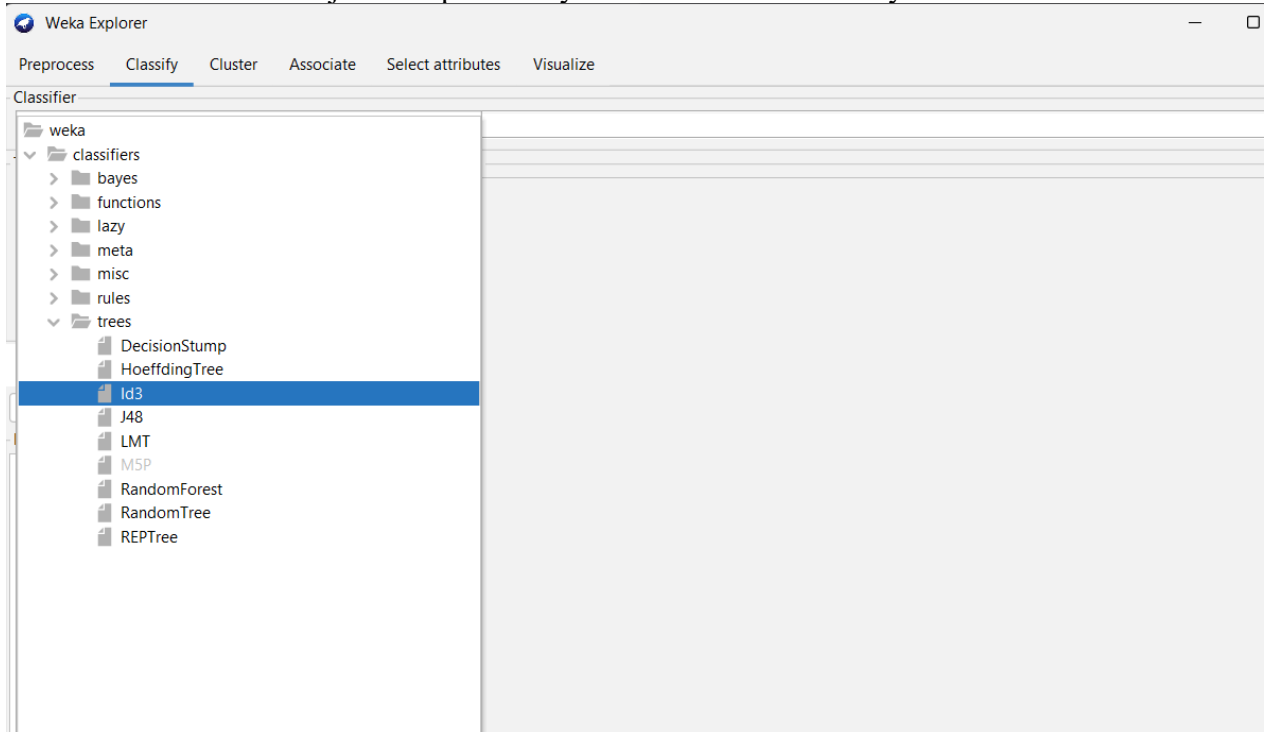


2. ID3 Classifier

- First install simpleEducationalLearningSchemes from package manager on the tools tab



- Now load the dataset just like previously and select ID3 in classify



- Run it with cross validation fold as 4

The screenshot shows the Weka Explorer interface with the 'Classifier' tab. The 'Id3' classifier is selected. The 'Test options' section shows 'Cross-validation' selected with 'Folds' set to 4. The 'Classifier output' section displays the results of the cross-validation.

Classifier output

```
Age = Middle_Aged: Yes
Age = Senior
| Credit_Rating = Fair: Yes
| Credit_Rating = Excellent: No
```

Time taken to build model: 0.03 seconds

=== Stratified cross-validation ===

=== Summary ===

Metric	Value	Percentage
Correctly Classified Instances	2	20 %
Incorrectly Classified Instances	6	60 %
Kappa statistic	-0.2632	
Mean absolute error	0.75	
Root mean squared error	0.866	
Relative absolute error	195.0867	%
Root relative squared error	199.2113	%
Unclassified Instances	2	20 %
Total Number of Instances	10	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.667	1.000	0.286	0.667	0.400	-0.488	0.333	0.343	No
	0.000	0.333	0.000	0.000	0.000	-0.488	0.375	0.600	Yes
Weighted Avg.	0.250	0.583	0.107	0.250	0.150	-0.488	0.359	0.504	

=== Confusion Matrix ===

```
a b  <-- classified as
2 1 | a = No
5 0 | b = Yes
```

Discussion:

The analysis of ID3 and J48 algorithms on loan approval classification problem indicates that J48 clearly outperforms ID3 in terms of the predictive accuracy, having correctly classified 50% of observations against 20% by ID3 under 4-fold cross-validation. An important observation resulting from this experiment is that ID3 failed to classify 20% of observations due to its inability to deal with certain combinations of attributes that were not encountered in the tree during training stage but appeared in the test fold. J48 managed to classify all observations, illustrating an obvious superiority of the algorithm based on C4.5 algorithm with pruning technique. Despite the easy-to-understand textual representation of the decision tree created by ID3 for "Middle_Aged" and "Senior" categories, the algorithm is less efficient than J48 due to the latter's robustness in the case and a much higher Kappa statistic.

Conclusion:

In this experiment, we were able to see the difference that exists between ID3 and J48 in Weka. The better classifier was the J48 as it is more accurate and allows continuous attributes. It also has an internal pruning mechanism. The ability to visualize the tree in Weka helped to understand how the classifier arrived at its decisions.

Lab Number: 5

Title: Create a Decision Tree Model Using Python Programming Language

Objective: To build and evaluate a decision tree classification model programmatically using Python, and to visualize the resulting decision tree.

IDE/Tools Used: Python (scikit-learn, Pandas, Matplotlib, networkx), Jupyter Notebook

Theory:

The Decision Tree algorithm is a supervised machine learning technique used for classification and regression tasks. It works by recursively splitting the dataset into subsets based on the attribute that provides the highest information gain or lowest Gini impurity. The ID3 algorithm, which uses entropy and information gain, is one of the foundational methods for constructing decision trees. Entropy measures the uncertainty in the dataset, and information gain quantifies the reduction in entropy after a split. The attribute with the highest information gain is chosen as the root or internal node, and the process continues recursively until all data points are classified or no further splits are possible.

Decision trees are easy to interpret and visualize, making them a popular choice for decision-making tasks. However, they are prone to overfitting, especially with noisy data, and may require pruning or setting a maximum depth to improve generalization.

Information Gain and Entropy

The essence of the ID3 algorithm lies in quantifying the purity of data. To achieve this goal, Entropy, a metric that reflects the uncertainty or disorder of the system, is applied. The maximum value for Entropy occurs when a dataset contains an equal proportion of classes, while its minimum is equal to zero if all the samples belong to a single category. The Information Gain, which measures the entropy reduction after splitting, is calculated for each attribute, and the ID3 algorithm selects the feature with the largest value.

Recursive Construction

ID3 uses a top-down induction process where the tree grows recursively. The data set is first divided into partitions based on the values of the chosen root attribute. Then, the algorithm works its way down the tree until either all instances in the current partition are of the same class or there are no more attributes to consider. ID3 always selects the attribute that provides the most information at each step.

Implementation:

Dataset used:

	A	B	C	D	E	F	G
1	Goal_Scored	Goals_Conceded	Possession	Home	Away	Opponent	Result
2	High	Low	High	Yes	No	High	Win
3	Medium	Medium	Medium	Yes	No	High	Win
4	Low	High	Low	No	Yes	High	Lose
5	Medium	Low	High	Yes	No	Medium	Win
6	Low	High	Low	No	Yes	Medium	Lose
7	High	Low	Medium	No	Yes	Low	Win
8	Medium	Medium	Medium	Yes	No	Low	Win
9	Low	Low	High	No	Yes	Low	Win
10	Low	Medium	High	Yes	No	Low	Win
11	High	Low	Medium	No	Yes	High	Win
12	Low	High	Low	No	Yes	High	Lose
13	Medium	Medium	High	Yes	No	Medium	Win
14	High	Medium	Medium	Yes	No	High	Win
15	Low	Medium	Medium	Yes	No	Medium	Lose
16	Medium	Low	High	No	Yes	Low	Win

```
import json
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# Define Functions for ID3 Algorithm
def entropy(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    probs = labels.value_counts() / n
    return -np.sum(probs * np.log2(probs))

def information_gain(df, feature, target):
    parent_entropy = entropy(df[target])
    weighted_child = sum(
        (len(subset) / len(df)) * entropy(subset[target])
        for _, subset in df.groupby(feature)
    )
    return parent_entropy - weighted_child

def best_feature(df, features, target):
    gains = {f: information_gain(df, f, target) for f in features}
    print("Information Gains:", {k: round(v, 4) for k, v in gains.items()})
    return max(gains, key=gains.get)
```

```

def ID3(df, features, target):
    if len(df[target].unique()) == 1:
        return df[target].iloc[0]

    if not features:
        return df[target].mode()[0]

    best = best_feature(df, features, target)
    tree = {best: {}}

    for value, subset in df.groupby(best):
        tree[best][value] = ID3(
            subset,
            [f for f in features if f != best],
            target,
        )

    return tree

# Load Dataset
dir_path = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(dir_path, "barcelona_ucl_dataset.csv"))
TARGET = "Result"

# Build Decision Tree
tree = ID3(df, [col for col in df.columns if col != TARGET], TARGET)
print("Decision Tree:", json.dumps(tree, indent=2))

# Build Recursive Tree Layout
def subtree_width(node):
    if not isinstance(node, dict):
        return 1
    root = list(node.keys())[0]
    return sum(subtree_width(child) for child in node[root].values())

def add_nodes(graph, node, parent=None, edge_label="", x=0, y=0):
    if isinstance(node, dict):
        root = list(node.keys())[0]
        node_id = f"node_{len(graph.nodes())}"
        graph.add_node(node_id, label=root)
        graph.nodes[node_id]["pos"] = (x, y)
        if parent is not None:
            graph.add_edge(parent, node_id, label=edge_label)

    children = node[root]
    total_width = sum(subtree_width(child) for child in children.values())
    start_x = x - total_width / 2
    offset = start_x
    for branch, child in children.items():
        width = subtree_width(child)
        child_x = offset + width / 2

```

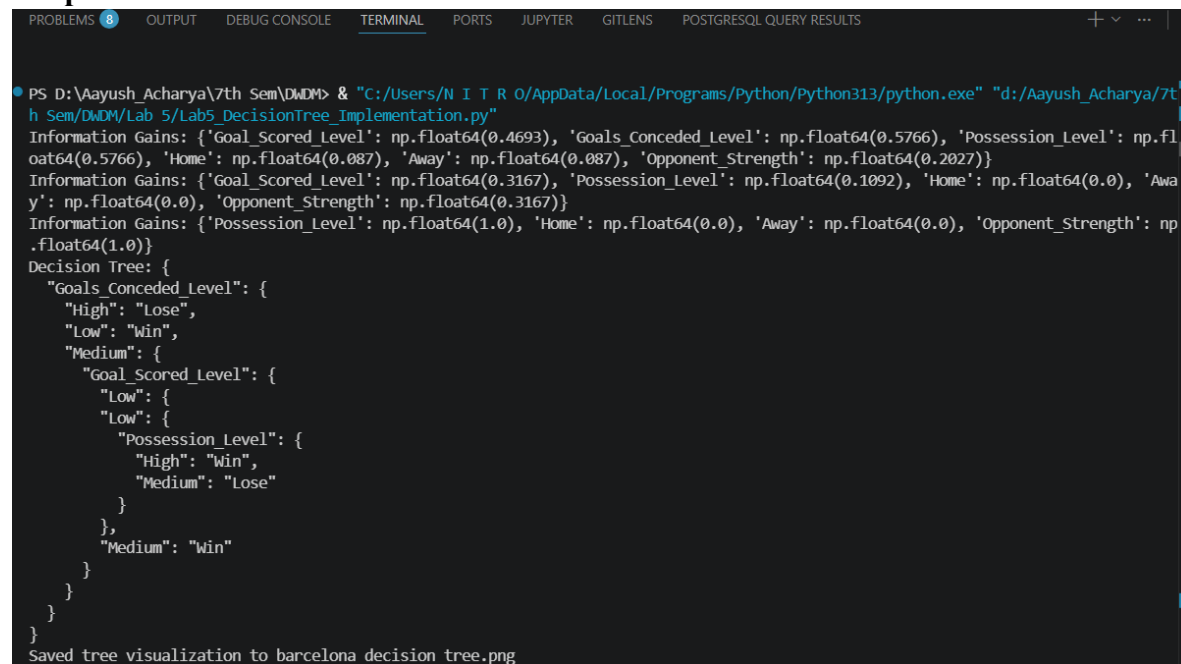
```

        add_nodes(graph, child, parent=node_id, edge_label=branch, x=child_x, y=y - 2)
        offset += width
    return node_id
else:
    leaf_id = f"leaf_{len(graph.nodes())}"
    graph.add_node(leaf_id, label=node)
    graph.nodes[leaf_id]["pos"] = (x, y)
    if parent is not None:
        graph.add_edge(parent, leaf_id, label=edge_label)
    return leaf_id

def collect_positions(graph):
    return {n: data["pos"] for n, data in graph.nodes(data=True)}

```

Output:



```

PS D:\Aayush_Acharya\7th Sem\DWDM> & "C:/Users/N I T R O/AppData/Local/Programs/Python/Python313/python.exe" "d:/Aayush_Acharya/7th Sem/DWDM/Lab 5/Lab5_DecisionTree_Implementation.py"
Information Gains: {'Goal_Scored_Level': np.float64(0.4693), 'Goals_Conceded_Level': np.float64(0.5766), 'Possession_Level': np.float64(0.5766), 'Home': np.float64(0.087), 'Away': np.float64(0.087), 'Opponent_Strength': np.float64(0.2027)}
Information Gains: {'Goal_Scored_Level': np.float64(0.3167), 'Possession_Level': np.float64(0.1092), 'Home': np.float64(0.0), 'Away': np.float64(0.0), 'Opponent_Strength': np.float64(0.3167)}
Information Gains: {'Possession_Level': np.float64(1.0), 'Home': np.float64(0.0), 'Away': np.float64(0.0), 'Opponent_Strength': np.float64(1.0)}
Decision Tree: {
  "Goals_Conceded_Level": {
    "High": "Lose",
    "Low": "Win",
    "Medium": {
      "Goal_Scored_Level": {
        "Low": {
          "Low": {
            "Possession_Level": {
              "High": "Win",
              "Medium": "Lose"
            }
          },
          "Medium": "Win"
        }
      }
    }
  }
}
Saved tree visualization to barcelona_decision_tree.png

```

```

g = nx.DiGraph()
add_nodes(g, tree, x=0, y=0)
pos = collect_positions(g)
labels = nx.get_node_attributes(g, "label")
edge_labels = nx.get_edge_attributes(g, "label")

plt.figure(figsize=(14, 9))
nx.draw_networkx_nodes(g, pos, node_color="#5B9BD5", node_size=2800, node_shape="s")
nx.draw_networkx_labels(g, pos, labels, font_size=10, font_color="white", font_weight="bold")
nx.draw_networkx_edges(g, pos, arrows=True, arrowsize=20, edge_color="#555555", width=1.8)
for (u, v), label in edge_labels.items():
    x1, y1 = pos[u]
    x2, y2 = pos[v]
    plt.text((x1 + x2) / 2, (y1 + y2) / 2, label,
             fontsize=9, color="#333333",
             bbox=dict(boxstyle="round,pad=0.2", fc="lightyellow", ec="gray", alpha=0.8),
             ha="center", va="center")

```

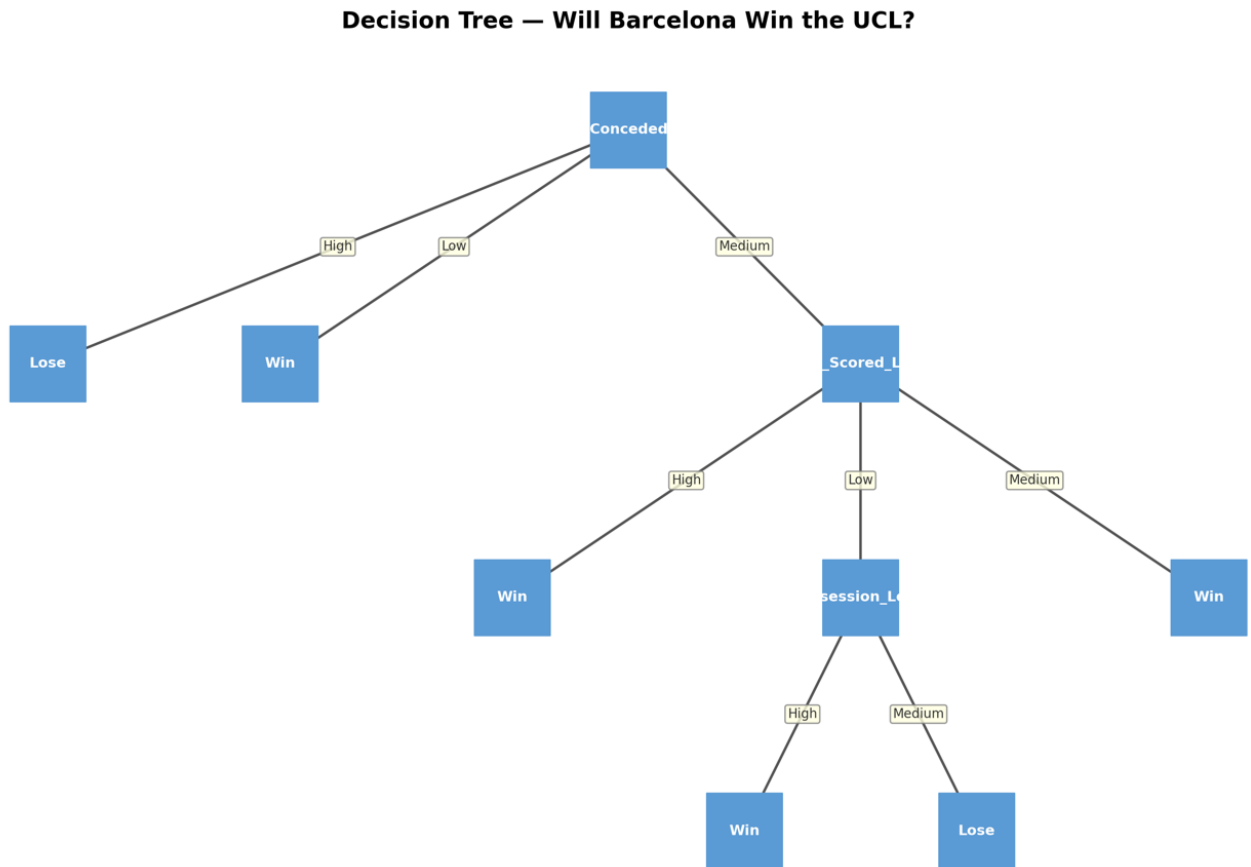


```

plt.title("Decision Tree — Will Barcelona Win the UCL?", fontsize=16, fontweight="bold", pad=20)
plt.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(dir_path, "barcelona_decision_tree.png"), dpi=220)
plt.close()
print("Saved tree visualization to barcelona_decision_tree.png")

```

Output:



Discussion:

The ID3 implementation used Barcelona-style match features and selected splits by information gain. The tree first splits on Goals_Conceded_Level, which is the most discriminative attribute, then refines medium concession cases with Goal_Scored_Level and Possession_Level. The resulting structure clearly shows how the algorithm separates wins and losses based on match context rather than raw scores alone.

Conclusion:

In summary, this lab produced a working ID3 decision tree that learns from a football dataset and saves a visual tree image. The model demonstrates how categorical match signals can be combined to classify Barcelona outcomes, and the saved visualization makes the decision path easy to inspect.

Lab Number: 6

Title: Illustrate Multilayer Perceptron in WEKA

Objective: To understand and demonstrate the working of a Multilayer Perceptron (MLP) neural network using the Weka tool on a suitable dataset.

IDE/Tools Used: MS Excel, Weka

Theory:

Artificial Neural Network

Artificial Neural Network (ANN) is a computational model inspired by the brain, where many simple processing units called neurons are arranged in layers and connected by weighted links. The network learns by adjusting those weights so that the output produced for each input pattern matches the expected result. ANNs are especially useful for capturing non-linear relationships in data, because each neuron applies a non-linear activation function to a weighted sum of its inputs.

Multilayer Perceptron (MLP)

The Multilayer Perceptron (MLP) is a type of feedforward ANN with at least one hidden layer between the input and output layers. Each neuron in one layer is fully connected to every neuron in the next layer. The hidden layer allows the MLP to learn complex decision boundaries, which a single-layer perceptron cannot represent. In Weka, MLP is implemented as a configurable neural network where the number of hidden neurons, learning rate, momentum, and training cycles can be adjusted.

Backpropagation

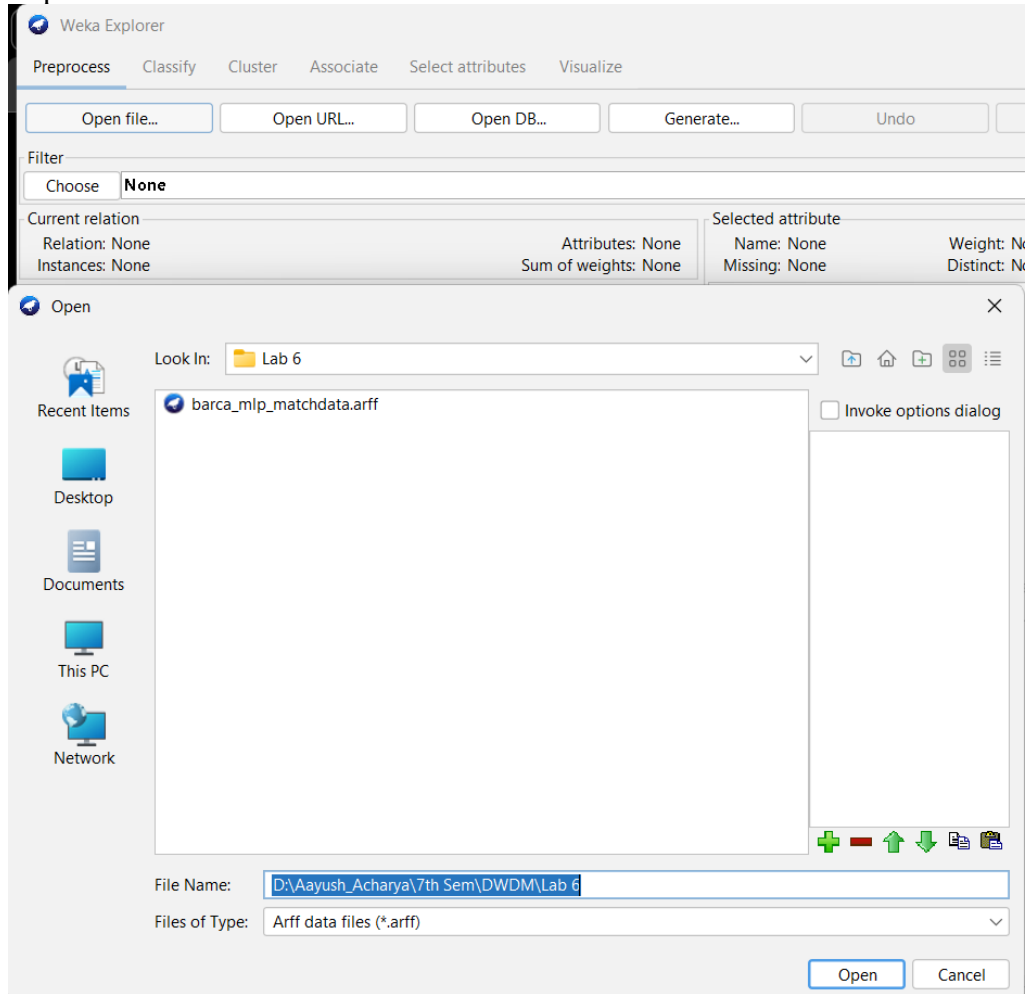
The training of MLPs during forward propagation, input values pass through the network and produce an output. The difference between the predicted output and the actual label is measured by a loss function. Backpropagation then moves this error backward through the network, layer by layer, computing gradients of the loss with respect to each weight. The weights are updated using gradient descent, which reduces the error iteratively and lets the network learn from examples.

MLP in Weka

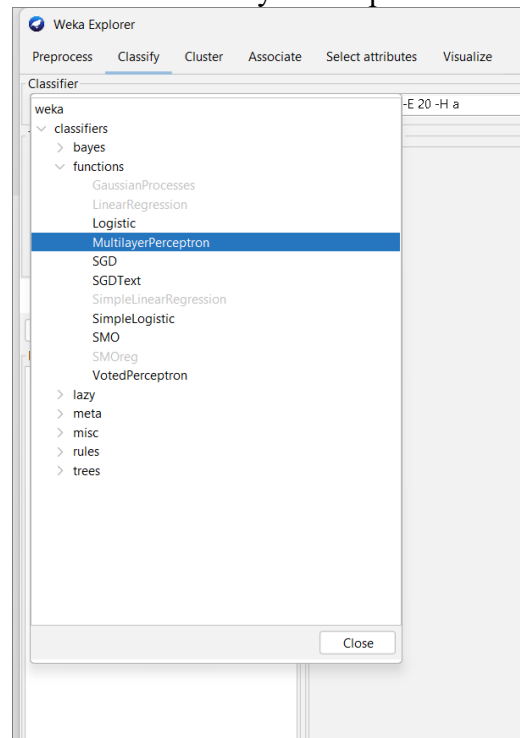
The MLP algorithm in Weka can be accessed through the Classify tab under functions > MultilayerPerceptron. One can modify parameters such as the number of hidden layers, number of neurons, learning rate, momentum, number of training cycles, and validation set percentage. It also offers a visualization tool for depicting the structure of the network during training.

Implementation

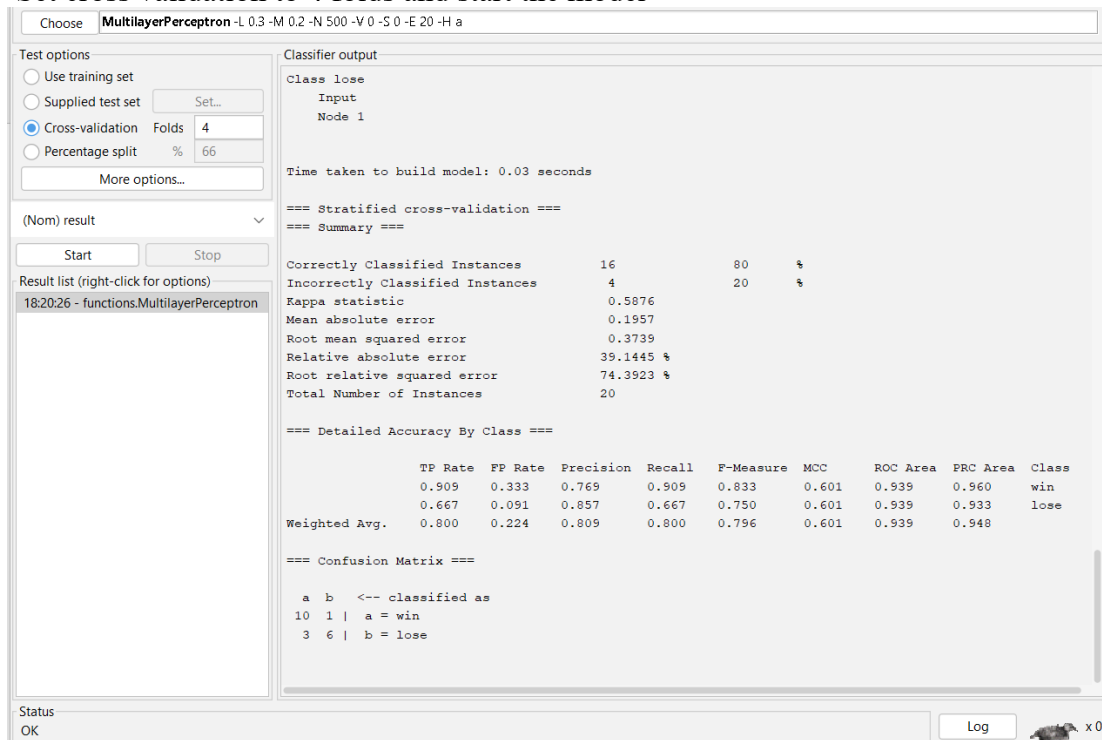
- Open the dataset on weka



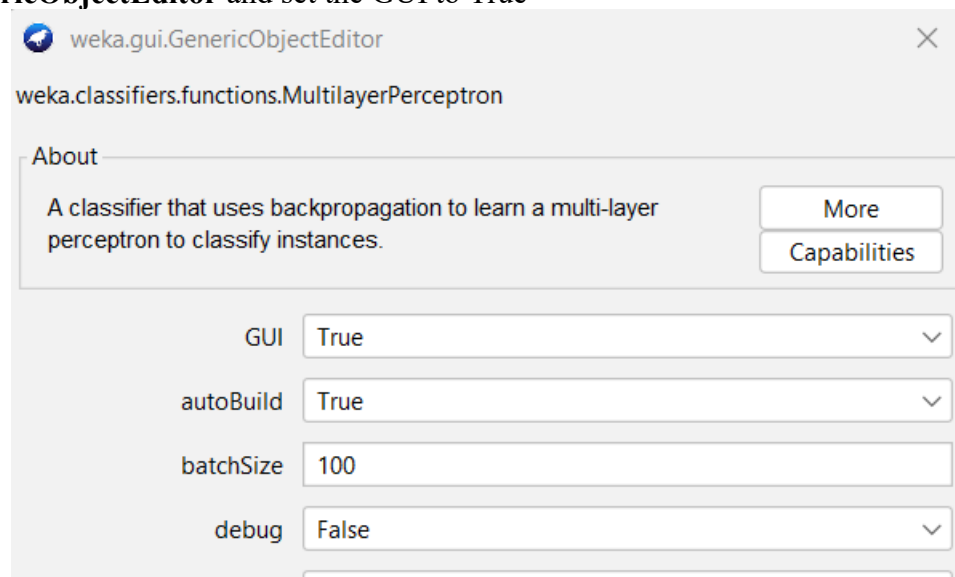
- Select The MultilayerPerceptron inside Classify tab



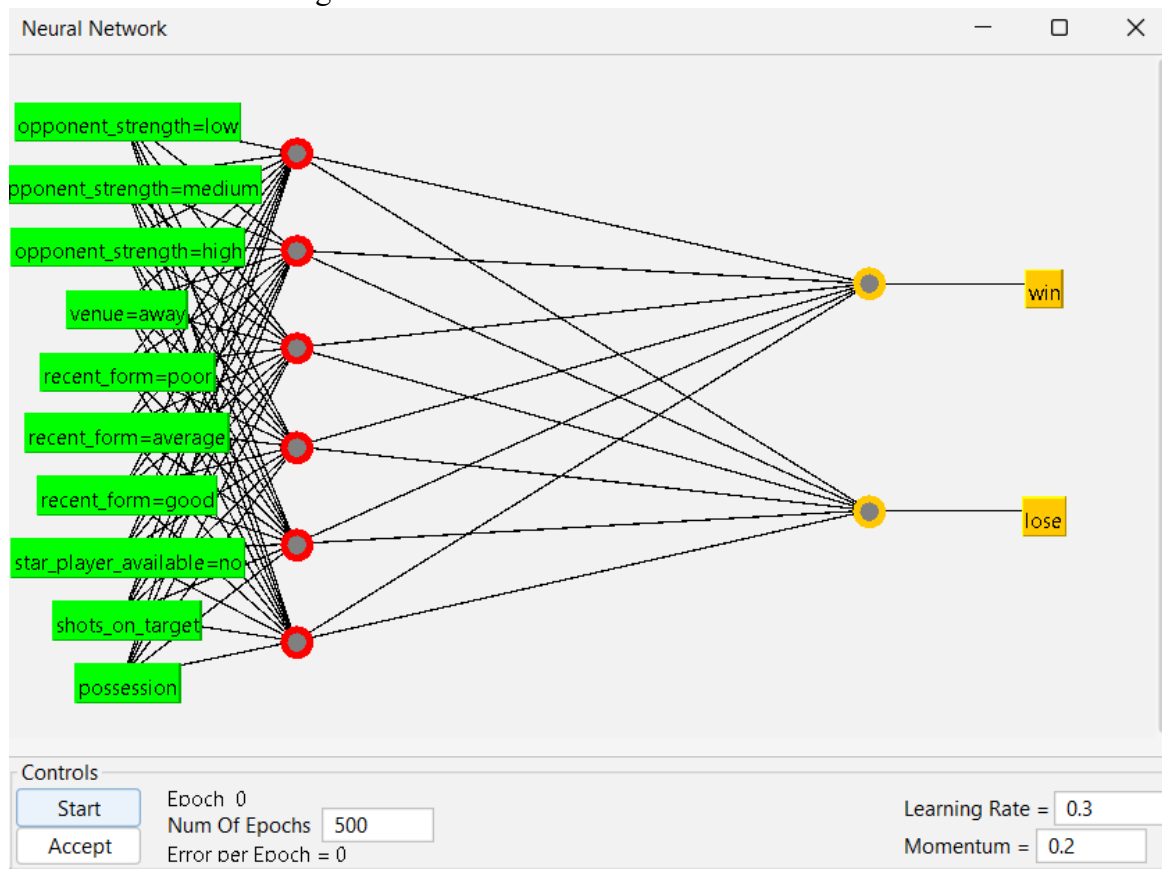
- Set cross validation to 4 folds and start the model



- Click on the text box next to "Choose" (where it says MultilayerPerceptron...) to open the **GenericObjectEditor** and set the GUI to True



- Run the model again for the visualization



Discussion:

This lab demonstrated MLP training in Weka using a football match dataset designed around Barcelona outcomes. The MLP model learned from match conditions such as opponent strength, venue, form, star player availability, shots on target, and possession. Using cross-validation ensured the model performance was evaluated reliably. The training process showed how the network weights were adjusted via backpropagation to separate win and lose cases, and the Weka visualization helped make the network structure more tangible.

Conclusion:

The experiment succeeded in illustrating how Multilayer Perceptron works in Weka. It showed that MLP can model non-linear relationships in football match data, and that careful configuration of hidden layers, learning rate, and training cycles is important for good performance. The lab also made the neural network workflow in Weka clear, from loading the dataset and selecting MLP to running cross-validation and visualizing the trained network.

Lab Number: 7

Title: Frequent Pattern Mining Using Apriori and FPGrowth with Minimum Support 0.3 and Strong Association Rules with Minimum Confidence 0.85

Objective: To mine frequent itemsets in Weka using Apriori and FPGrowth with minimum support 0.3 and extract strong association rules from Barcelona match-day fan purchases at minimum confidence 0.85.

IDE/Tools Used: Weka, MS Excel

Theory:

Association Rule Mining

Association rule mining discovers useful relationships in transaction-style data. A rule has the form $X \rightarrow Y$, where X and Y are itemsets. Support measures how often the combined itemset appears in the dataset, while confidence measures how often the rule is correct when X occurs. Lift compares the observed co-occurrence of X and Y to what would be expected if they were independent.

Apriori Algorithm

Apriori finds frequent itemsets by generating and pruning candidates in a level-wise manner. It uses the anti-monotonicity property: if an itemset is not frequent, none of its supersets can be frequent.

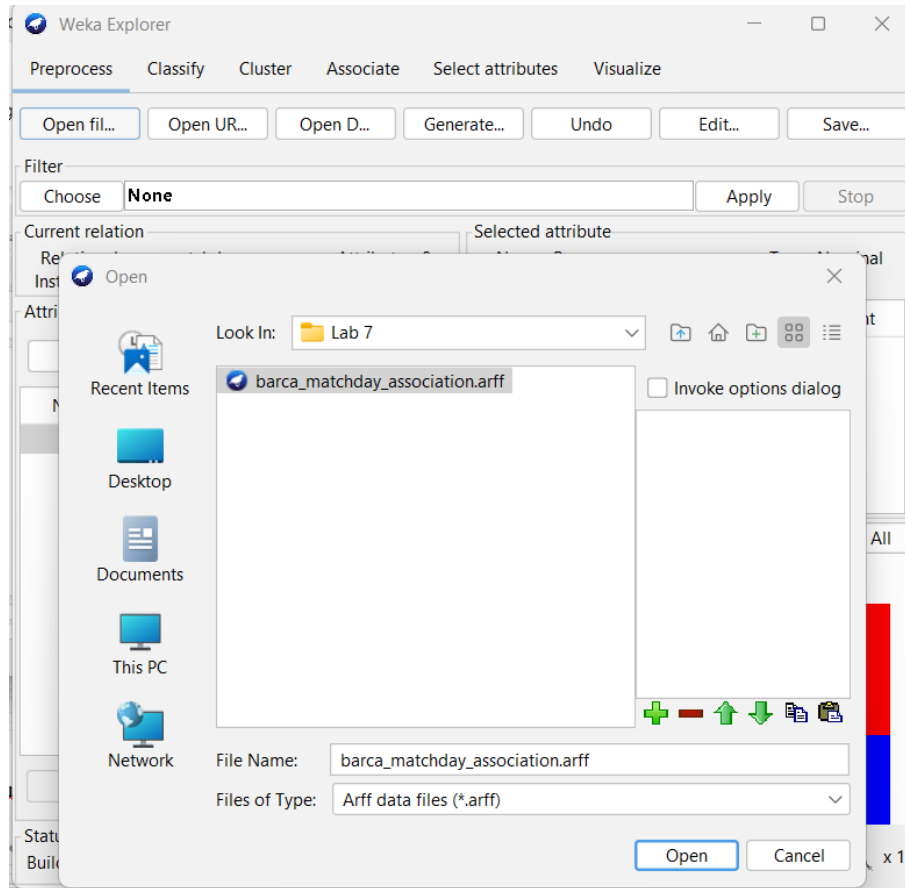
FPGrowth Algorithm

FPGrowth is a faster alternative that compresses the data into an FP-tree and mines frequent patterns directly without generating candidate sets.

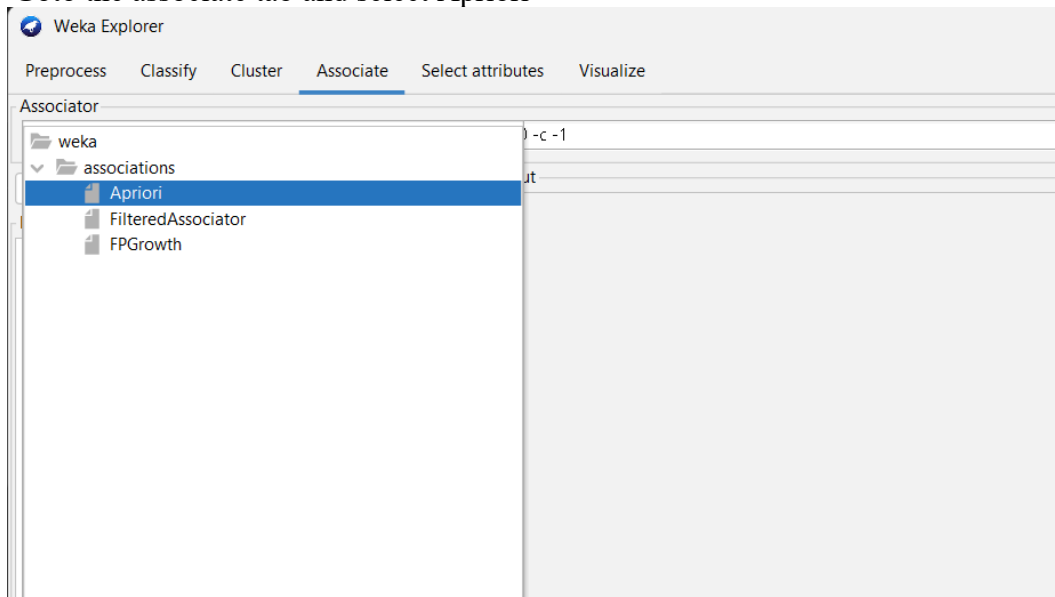
Implementation:

1. Using Apriori algorithm

- Open the dataset



- Goto the associate tab and select Apriori



- Goto the filter and set lowerBoundMinSupport to 0.3 and minMetric to 0.85

weka.gui.GenericObjectEditor

weka.associations.Apriori

About

Class implementing an Apriori-type algorithm. More Capabilities

car

classIndex

delta

doNotCheckCapabilities

lowerBoundMinSupport

metricType

minMetric

numRules

outputItemSets

removeAllMissingCols

significanceLevel

treatZeroAsMissing

upperBoundMinSupport

verbose

Open... Save... OK Cancel

- Now click the start button

Weka Explorer

Preprocess Classify Cluster **Associate** Select attributes Visualize

Associator

Choose **Apriori** -N 10 -T 0 -C 0.85 -D 0.05 -U 1.0 -M 0.3 -S -1.0 -c -1

Start Stop

Result list (right-click for ...)

18:39:25 - Apriori

Associator output

```
Size of set of large itemsets L(1): 10
Size of set of large itemsets L(2): 17
Size of set of large itemsets L(3): 14
Size of set of large itemsets L(4): 6
Size of set of large itemsets L(5): 1

Best rules found:
1. SnackBar=f 17 ==> Popcorn=t 17 <conf:(1)> lift:(1.18) lev:(0.13) [2] conv:(2.55)
2. Popcorn=t 17 ==> SnackBar=f 17 <conf:(1)> lift:(1.18) lev:(0.13) [2] conv:(2.55)
3. SnackBar=f Camera=f 13 ==> Popcorn=t 13 <conf:(1)> lift:(1.18) lev:(0.1) [1] conv:(1.95)
4. Popcorn=t Camera=f 13 ==> SnackBar=f 13 <conf:(1)> lift:(1.18) lev:(0.1) [1] conv:(1.95)
5. Flag=f 12 ==> Camera=f 12 <conf:(1)> lift:(1.25) lev:(0.12) [2] conv:(2.4)
6. Scarf=f 11 ==> Jersey=f 11 <conf:(1)> lift:(1.82) lev:(0.25) [4] conv:(4.95)
7. Jersey=f 11 ==> Scarf=f 11 <conf:(1)> lift:(1.82) lev:(0.25) [4] conv:(4.95)
8. Beer=t 10 ==> Popcorn=t 10 <conf:(1)> lift:(1.18) lev:(0.08) [1] conv:(1.5)
9. Soda=f 10 ==> Beer=t 10 <conf:(1)> lift:(2) lev:(0.25) [5] conv:(5)
10. Beer=t 10 ==> Soda=f 10 <conf:(1)> lift:(2) lev:(0.25) [5] conv:(5)
```

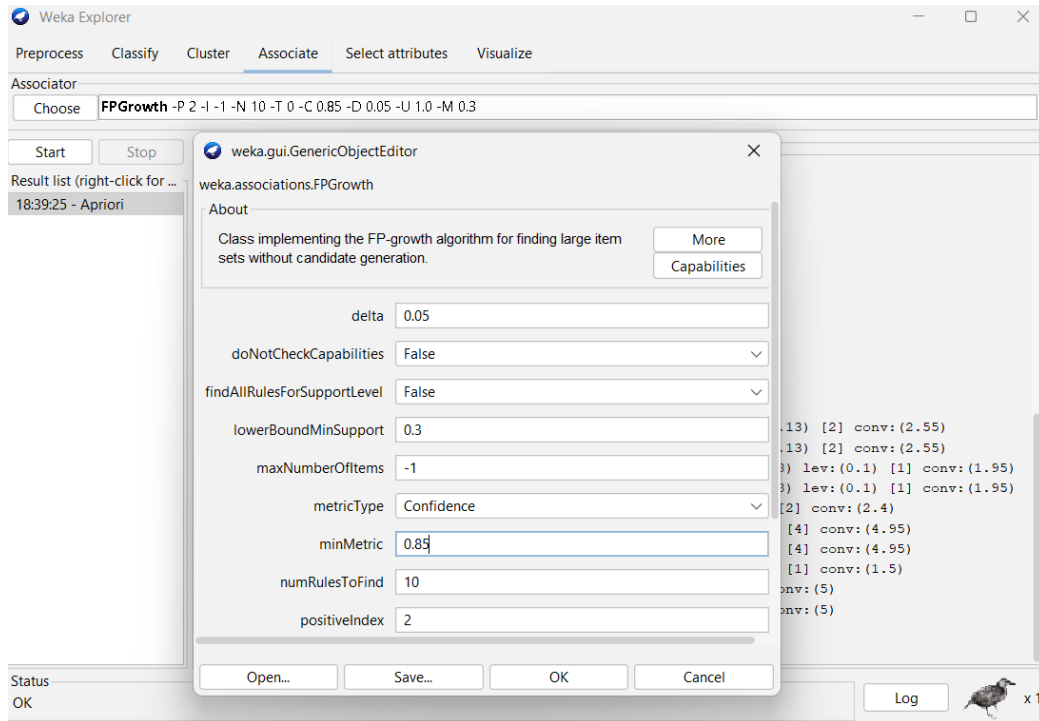
Status

OK

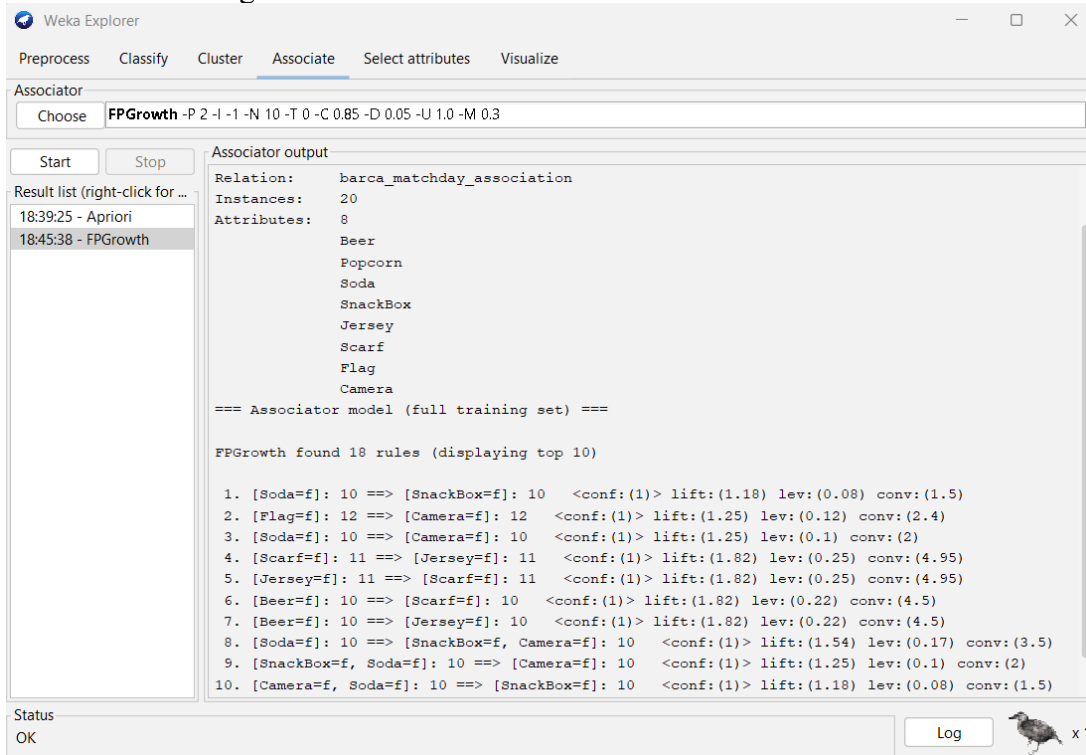
Log  x 1

2. Using FPGrowth algorithm

- Similar to the apriori algorithm select FPGrowth and set lowerBoundMinSupport to 0.3 and minMetric to 0.85



- Start the algorithm



Discussion:

Using the Barcelona match-day dataset, both Apriori and FPGrowth should find frequent fan purchase patterns that meet support 0.3 and confidence 0.85. The results can reveal common combinations such as fans who buy a jersey also often buy a scarf, or fans who buy beer are likely to purchase popcorn. FPGrowth should be faster because it uses the FP-tree structure and avoids generating many candidates, while Apriori uses candidate generation and iterative pruning.

Conclusion:

This lab demonstrates that Weka can mine strong association rules from a custom dataset using Apriori and FPGrowth. With the same thresholds, both algorithms should discover similar patterns, but FPGrowth is usually more efficient. The exercise reinforces the importance of support and confidence values in selecting meaningful rules, while showing how association mining works on fan purchase behavior rather than a classic grocery example

.

Lab Number: 8

Title: Programmatically Demonstrate a Binary Classifier Using Naive Bayes Algorithm

Objective: Implement and evaluate a binary classifier using the Naive Bayes algorithm on a custom dataset with Python.

IDE/Tools Used: Python (Pandas, Matplotlib), Jupyter Notebook

Theory:

Naive Bayes Algorithm

The Naive Bayes classifier applies Bayes' Theorem to compute the probability of a class given observed features, assuming conditional independence among features. In binary classification, the algorithm calculates posterior probabilities for the two classes and selects the class with the higher probability. Laplace smoothing avoids zero probabilities, and using log probabilities keeps the computation numerically stable.

Bayes' Theorem

Bayes' Theorem suggests that the posterior probability of any class taking into consideration the provided features is proportional to the likelihood of the features multiplied by the probability of the class itself. $P(C|X)$ is proportionate to $P(X|C) \times P(C)$. Here C is a class, while X stands for a vector of features. In the Naive Bayes classifier, the likelihood can be calculated as a multiplication of feature probabilities.

Binary Classification

In binary classification, we classify the dataset into only two groups or classes, like spam or non-spam, diseased or healthy, and positive or negative. In the case of Naive Bayes binary classification, the algorithm calculates the posterior probabilities for both classes and assigns the higher one as the class label. The decision boundary is found from the comparison between the posterior probabilities of the two classes.

Types of Naive Bayes

Several versions of Naive Bayes algorithms exist based on different kinds of features. For example, the Gaussian Naive Bayes model assumes that continuous features have a Gaussian distribution. On the other hand, the Multinomial Naive Bayes classifier is applied when dealing with discrete count features, typically seen in text classification. Another variant is the Bernoulli Naive Bayes, which uses binary features (present or absent). The Python programming language implements these models via the `GaussianNB`, `MultinomialNB`, and `BernoulliNB` classes within the scikit-learn library.

Implementation:

```
import pandas as pd
import numpy as np
```

```
# Load dataset
```

```
base_dir = r"d:/Aayush_Acharya/7th Sem/DWDM/Lab 8"
```

```
df = pd.read_csv(f"{base_dir}/loan_application_data.csv")
```

```
def train_naive_bayes(df, target_col):
```

```
    total_docs = len(df)
```

```
    classes = df[target_col].unique()
```

```
    priors = {c: len(df[df[target_col] == c]) / total_docs for c in classes}
```

```
    features = [col for col in df.columns if col != target_col]
```

```
    model = {}
```

```
    for c in classes:
```

```
        model[c] = {}
```

```
        df_c = df[df[target_col] == c]
```

```
        for feat in features:
```

```
            values = df_c[feat].unique()
```

```
            model[c][feat] = {
```

```
                val: (len(df_c[df_c[feat] == val]) + 1) / (len(df_c) + len(df[feat].unique()))
```

```
                for val in df[feat].unique()
```

```
            }
```

```
    return priors, model, classes
```

```
def predict(priors, model, classes, sample):
```

```
    probs = {}
```

```
    for c in classes:
```

```
        prob = np.log(priors[c])
```

```
        for feat, val in sample.items():
```

```
            prob += np.log(model[c][feat].get(val, 1e-6))
```

```
        probs[c] = prob
```

```
    return max(probs, key=probs.get)
```

```
priors, model, classes = train_naive_bayes(df, 'Approved')
```

```
test_samples = df.drop(columns=['Approved']).to_dict(orient='records')
```

```
actuals = df['Approved'].tolist()
```

```
predictions = [predict(priors, model, classes, s) for s in test_samples]
```

```
accuracy = sum(1 for p, a in zip(predictions, actuals) if p == a) / len(actuals)
```

```
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

```
new_application = {'Has_Credit_Card': 1, 'Employed': 1, 'High_Income': 0, 'Good_Credit_History': 1}
```

```
prediction = predict(priors, model, classes, new_application)
```

```
print(f"Prediction for new application {new_application}: {'Approved' if prediction == 1 else 'Not Approved'}")
```

Dataset:

	A	B	C	D	E
1	Has_Credit_Card	Employed	High_Income	Good_Credit	Approved
2	1	1	1	1	1
3	1	1	0	1	1
4	0	1	1	1	1
5	1	0	1	1	1
6	0	1	0	1	0
7	1	0	0	0	0
8	0	0	1	0	0
9	1	1	1	0	1
10	0	0	0	1	0
11	1	1	0	0	1
12	0	1	1	0	1
13	1	0	1	0	1
14	0	1	0	0	0
15	1	1	1	1	1
16	0	0	1	1	0
17	1	0	0	1	0
18	0	1	1	1	1
19	1	0	1	1	1
20	0	0	0	1	0
21	1	1	0	1	1

Output:

Model Accuracy: 100.00%

Prediction for new application {'Has_Credit_Card': 1, 'Employed': 1, 'High_Income': 0, 'Good_Credit_History': 1}: Approved

Discussion:

A Naive Bayes classifier was trained on a loan approval dataset with binary applicant features. Class priors and feature likelihoods were computed with Laplace smoothing, and log probabilities were used to avoid underflow. The model predicted approval outcomes for all examples and also evaluated a new loan application.

Conclusion:

Naive Bayes proved to be a fast and effective binary classifier for this dataset. Despite the simplifying assumption of feature independence, the model is useful for quick classification tasks and can handle binary feature data naturally.

Lab Number: 9

Title: Implement K-Means Clustering, Agglomerative Hierarchical Clustering, and DBScan Using Python

Objective: Understand and implement K-Means, Agglomerative Hierarchical Clustering, and DBSCAN with Python, then visualize the results.

IDE/Tools Used: Python (Pandas, Matplotlib, SciPy), Jupyter Notebook

Theory: Clustering

Clustering is an unsupervised method for grouping similar data points. K-Means partitions data into K clusters by minimizing intra-cluster variance. Agglomerative Hierarchical Clustering builds a cluster tree by successively merging the closest points or clusters. DBSCAN identifies dense regions of points as clusters and marks points in sparse regions as noise.

K-Means Clustering

The K-means is one of the widely adopted clustering methods. The algorithm divides data into K groups by first assigning points to the closest centroid then calculating new centroids based on the mean of points assigned to clusters. The process continues until there is minimal change in centroids. It works in minimizing the total intra-cluster variance (inertia). The user needs to specify the number of clusters, which determines the final results since their initial locations matter.

Agglomerative Hierarchical Clustering

The Agglomerative Hierarchical Clustering approach is a bottom-up method that begins with considering each data instance a different cluster and successively merges pairs of close clusters until there is one big cluster or a particular criterion is achieved. The clustering results are presented using a tree-structure plot known as the dendrogram, which illustrates how clusters were combined. The choice of different linkage metrics (single-linkage, complete-linkage, average-linkage, or Ward's linkage) defines how distances among clusters are computed. In the latter case, the sum of squared deviations from the mean of all clusters is minimized.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is a clustering algorithm that finds clusters of point instances which are dense, while identifying other areas where point distribution is sparse as outliers. DBSCAN uses the following two parameters: `eps`, which is the largest distance between two points for them to belong to a neighborhood of each other; and `min_samples` – the smallest size of dense point area or core points forming a neighborhood. Contrary to K-Means, the algorithm does not need to specify the desired number of clusters beforehand.

Implementation:

K-Means Clustering:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy
import dendrogram, linkage
import math
base_dir = r"d:/Aayush_Acharya/7th
Sem/DWDM/Lab 9"
df =
pd.read_csv(f"{base_dir}/sample_points.cs
v")
points = df[['x', 'y']].values

# K-Means implementation
def kmeans(points, k, max_iters=100):
    centroids = points[:k].copy()
    for _ in range(max_iters):
        clusters = [[] for _ in range(k)]
        for p in points:
            distances = [np.linalg.norm(p - c)
for c in centroids]
            clusters[np.argmin(distances)].appen
d(p)
        new_centroids = []
        for cluster in clusters:
            if cluster:
                new_centroids.append(np.mean(cl
uster, axis=0))
            else:
                new_centroids.append(centroids[l
en(new_centroids)])
        new_centroids =
np.array(new_centroids)
        if np.allclose(new_centroids,
centroids):
            break
        centroids = new_centroids
    return centroids, clusters

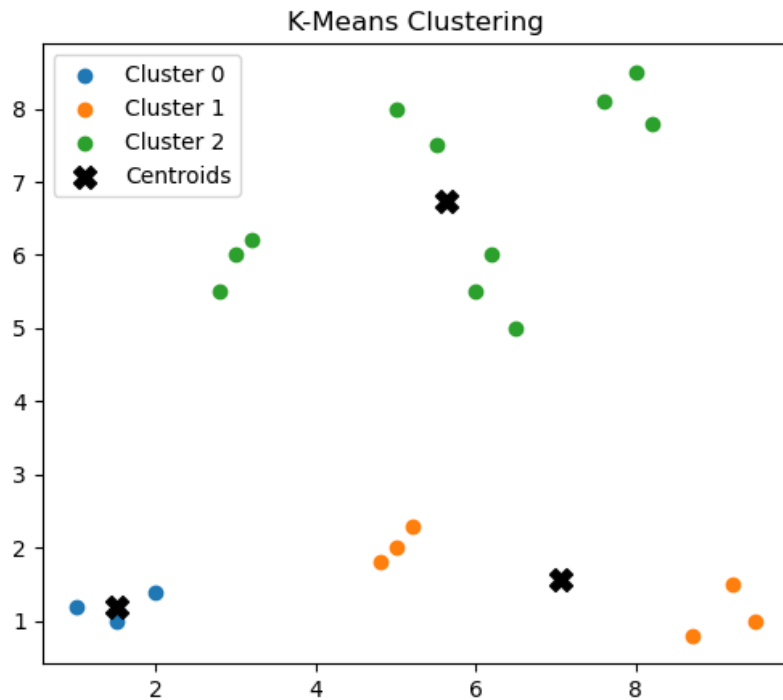
centroids, clusters = kmeans(points, k=3)
print('K-Means centroids:', centroids)
for i, cluster in enumerate(clusters):
    print(f'Cluster {i}:', len(cluster), 'points')

plt.figure(figsize=(6, 5))
for i, cluster in enumerate(clusters):
    cluster = np.array(cluster)
    if len(cluster) > 0:
        plt.scatter(cluster[:,0], cluster[:,1],
```

```
label=f'Cluster {i}')
plt.scatter(centroids[:,0], centroids[:,1],
c='black', marker='X', s=100,
label='Centroids')
plt.title('K-Means Clustering')
plt.legend()
plt.show()
```

Output:

```
K-Means centroids: [[1.5      1.2      ]
 [7.06666667 1.56666667]
 [5.63636364 6.73636364]]
Cluster 0: 3 points
Cluster 1: 6 points
Cluster 2: 11 points
```



Agglomerative Hierarchical Clustering:

```
# Agglomerative Hierarchical Clustering using SciPy
```

```
Z = linkage(points, method='ward')
```

```
plt.figure(figsize=(8, 4))
```

```
dendrogram(Z, labels=[f'P{i+1}' for i in range(len(points))], orientation='top', distance_sort='descending')
```

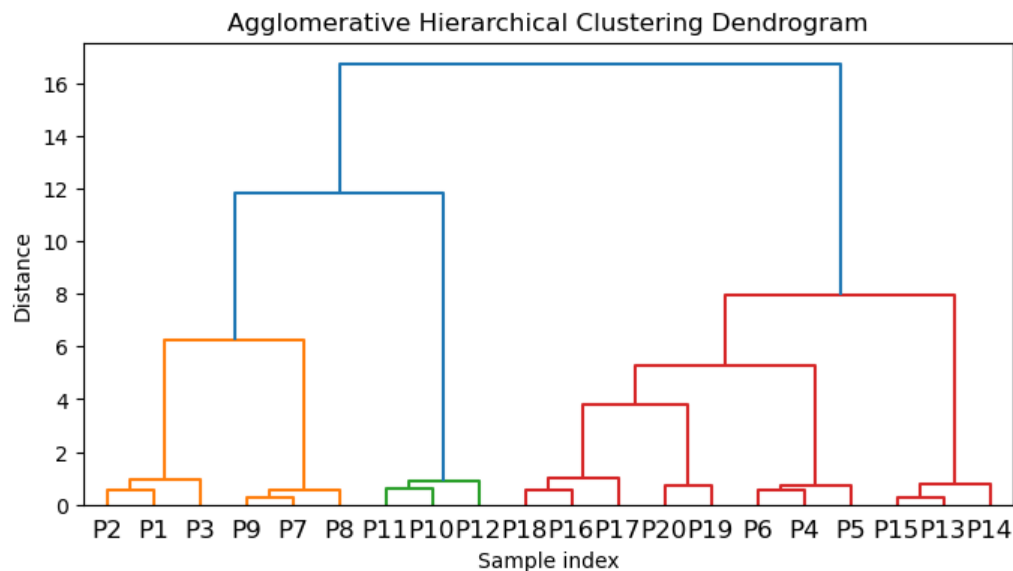
```
plt.title('Agglomerative Hierarchical Clustering Dendrogram')
```

```
plt.xlabel('Sample index')
```

```
plt.ylabel('Distance')
```

```
plt.show()
```

Output:



DBSCAN algorithm:

```
def dbscan(points, eps=1.5, min_pts=3):
```

```
    labels = [0] * len(points)
```

```
    cluster_id = 0
```

```
    def region_query(i):
```

```
        return [j for j, p in enumerate(points) if np.linalg.norm(points[i]-p) <= eps]
```

```
    for i in range(len(points)):
```

```
        if labels[i] != 0:
```

```
            continue
```

```
        neighbors = region_query(i)
```

```
        if len(neighbors) < min_pts:
```

```
            labels[i] = -1
```

```
            continue
```

```
        cluster_id += 1
```

```
        labels[i] = cluster_id
```

```
        seeds = [n for n in neighbors if n != i]
```

```
        while seeds:
```

```
            current = seeds.pop()
```

```
            if labels[current] == -1:
```

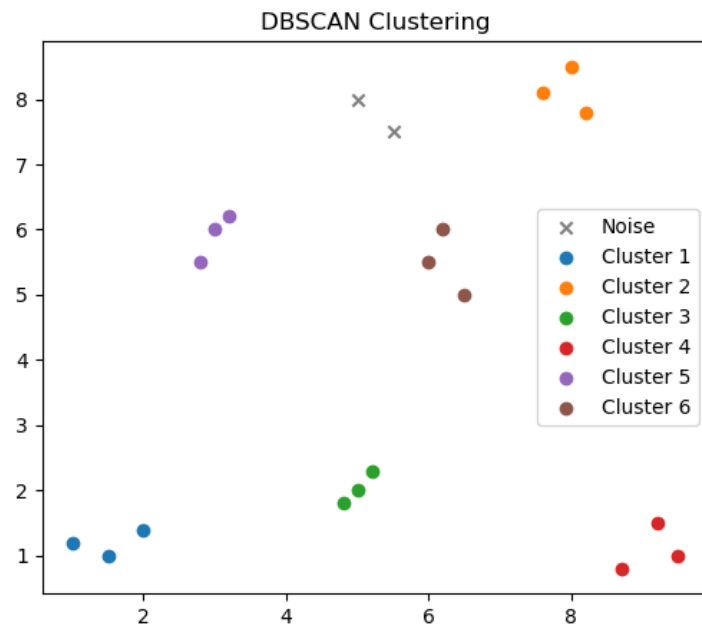
```
                labels[current] = cluster_id
```

```

    if labels[current] != 0:
        continue
    labels[current] = cluster_id
    current_neighbors = region_query(current)
    if len(current_neighbors) >= min_pts:
        seeds.extend([n for n in current_neighbors if labels[n] == 0])
    return labels
labels = dbscan(points, eps=1.5, min_pts=3)
unique_labels = sorted(set(labels))
plt.figure(figsize=(6, 5))
for label in unique_labels:
    cluster_points = points[[i for i, l in enumerate(labels) if l == label]]
    if label == -1:
        plt.scatter(cluster_points[:,0], cluster_points[:,1], c='gray', marker='x', label='Noise')
    else:
        plt.scatter(cluster_points[:,0], cluster_points[:,1], label=f'Cluster {label}')
plt.title('DBSCAN Clustering')
plt.legend()
plt.show()

```

Output:



Discussion:

The notebook demonstrates three clustering methods on a generic 2D point dataset. K-Means partitions the data into fixed clusters by iteratively updating centroids. Agglomerative clustering builds a dendrogram showing how data points merge into clusters. DBSCAN detects dense regions and identifies any points that do not belong to a cluster as noise.

Conclusion:

This lab illustrates how different clustering methods behave on the same dataset. K-Means works well for compact clusters when K is known. Agglomerative clustering provides a hierarchical structure without requiring a preset number of clusters. DBSCAN detects clusters of varying shape and separates noise automatically.

Lab Number: 10

Title: Apply Agglomerative Hierarchical Clustering Filter of WEKA on a Suitable Dataset

Objective: Apply the Agglomerative Hierarchical Clustering filter in Weka to a suitable dataset and analyze the resulting clusters.

IDE/Tools Used: Weka, MS Excel

Theory: Hierarchical Clustering

Hierarchical clustering constructs a hierarchy of clusters either by merging small clusters into larger ones or by dividing large clusters into smaller ones. In the agglomerative approach, each instance begins as its own cluster, and similar clusters are merged until the desired number of clusters or stopping criterion is reached. The result is often displayed as a dendrogram that shows the order and distance at which clusters were combined.

Agglomerative Approach

Agglomerative clustering can use different distance measures such as Euclidean or Manhattan distance. Linkage criteria determine how cluster distance is computed: single linkage uses the nearest neighbor, complete linkage uses the farthest points, average linkage uses the average distance, and Ward's method minimizes within-cluster variance.

Applying Hierarchical Clustering in Weka

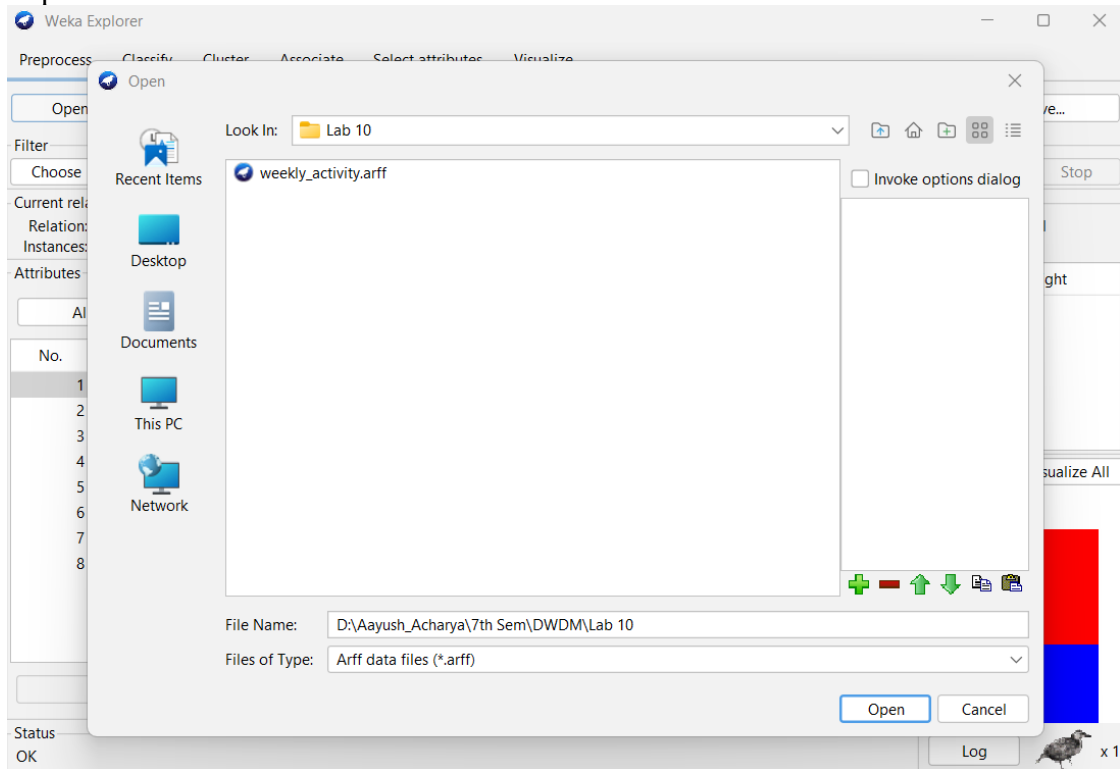
In Weka, agglomerative hierarchical clustering is performed with HierarchicalClusterer from the Cluster tab. You can choose the number of clusters with numClusters, select the linkage method such as SINGLE, COMPLETE, AVERAGE, or WARD, and choose the distance measure like EuclideanDistance. After running the algorithm, enable the tree or dendrogram output so Weka prints the hierarchy of merges and cluster distances. This allows you to study how the data points join step by step and how the model groups similar records.

Interpreting Clustering Results

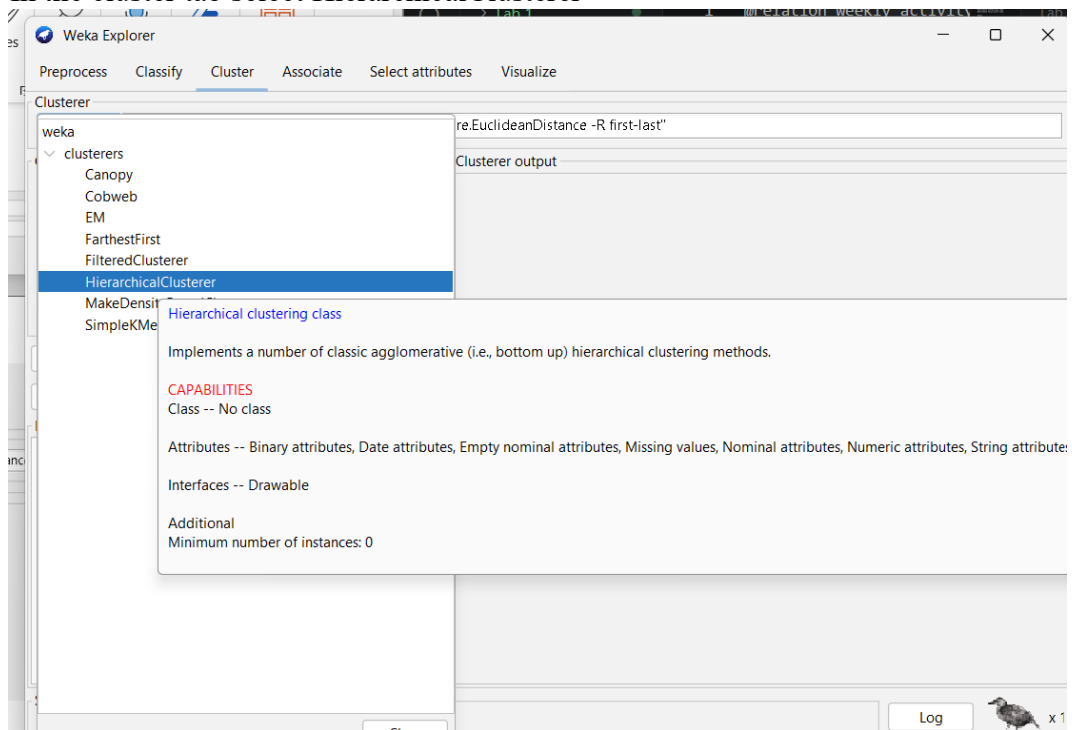
To interpret the results, examine how instances are grouped within each cluster and what characteristics those groups share. The Weka cluster summary shows the number of instances per cluster and how the data is distributed across clusters. Since clustering is unsupervised, there is no single correct answer, but you can evaluate the grouping by checking whether the clusters make sense in terms of the underlying data. Additional quality measures such as silhouette score, Davies-Bouldin index, or Dunn index can help compare cluster cohesion and separation when available.

Implementation

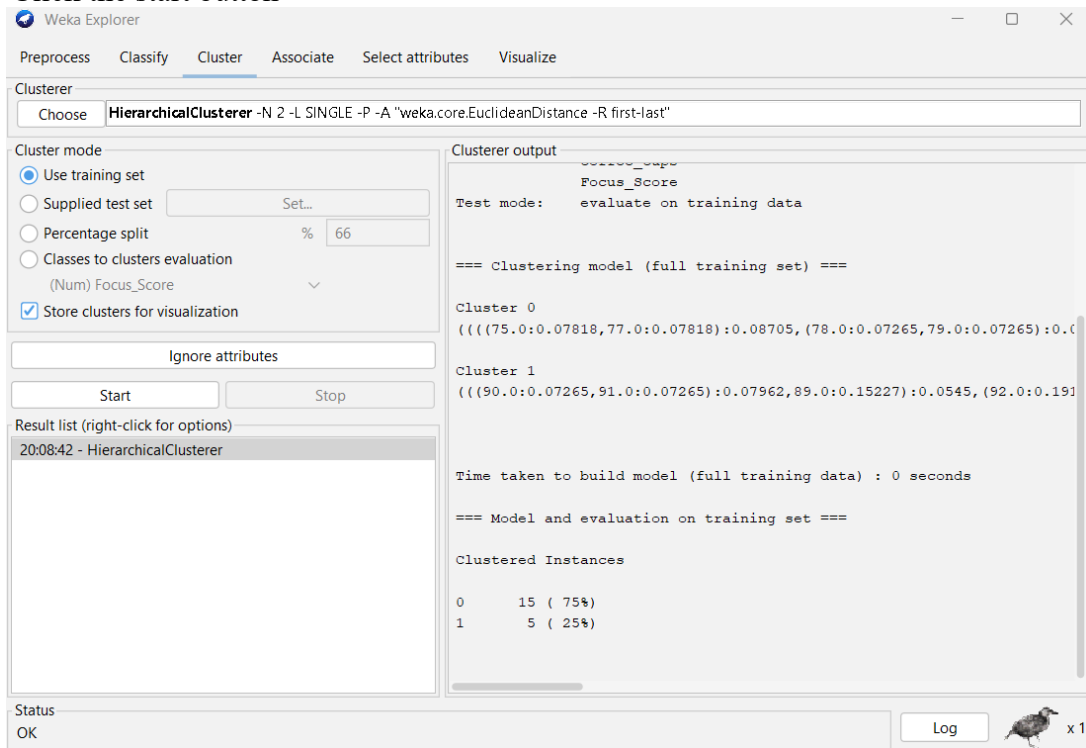
- Open the dataset



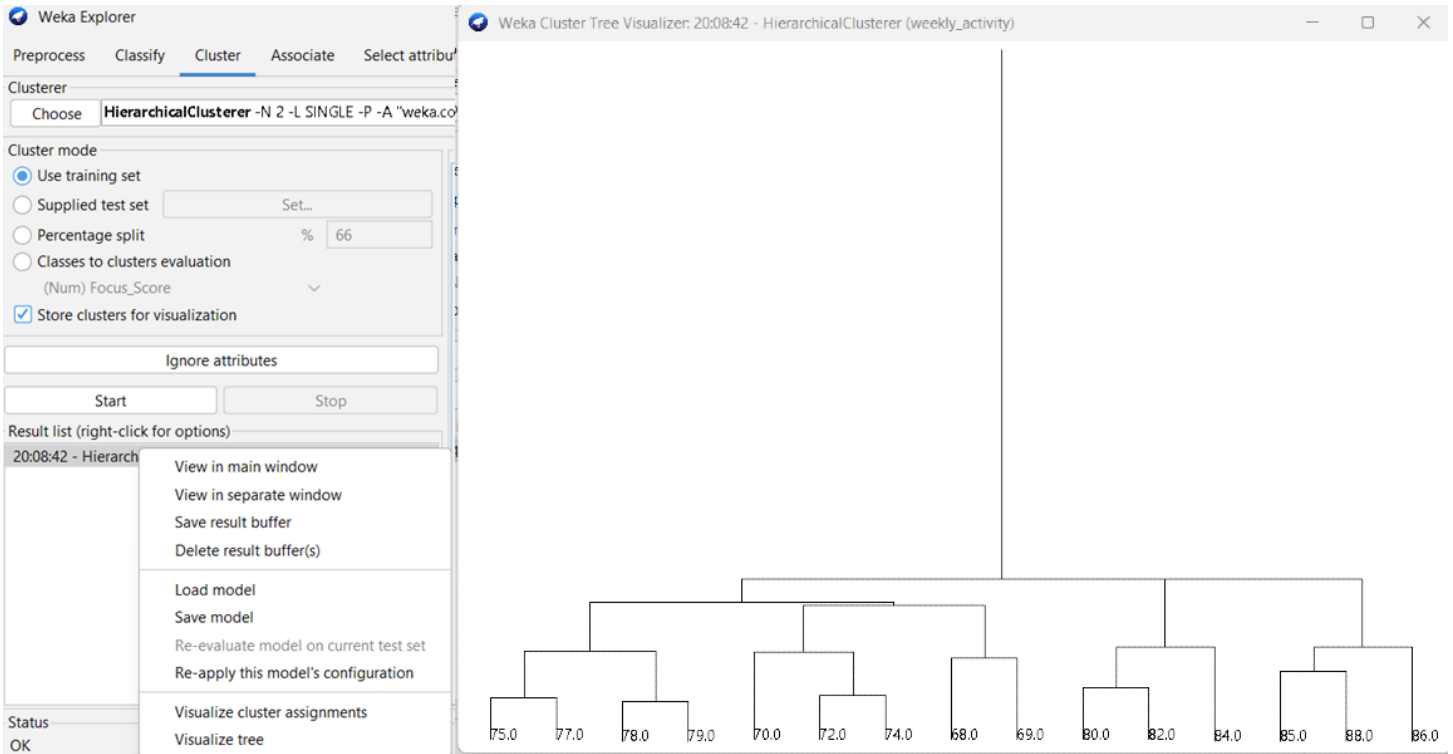
- In the cluster tab select HierarchicalClusterer



- Click the start button



- Right click on HirarchicalClusterer and select visualize tree



Discussion:

Agglomerative hierarchical clustering groups similar weekly activity profiles step by step. The dendrogram shows how individual weeks merge into clusters based on work hours, meetings, emails, coffee consumption, and focus score. The chosen linkage method affects the merge order and cluster shape. For example, single linkage may group points based on one pair of close samples, while complete linkage uses the farthest pair.

This procedure is useful when you want to explore the data structure without predefining exact cluster boundaries. Clusters can reveal patterns like high-workload weeks, low-activity weeks, and moderate weeks with balanced focus.

Conclusion:

This lab demonstrates Weka's HierarchicalClusterer for agglomerative clustering. The exercise shows how a dendrogram helps interpret cluster relationships and hierarchy. Agglomerative clustering is especially valuable when the dataset structure matters and when you want to compare different linkage methods and cluster distances.